

VEXA · Construcción de punta a punta

Guía de implementación. No certifica SaaS desplegado. Las fichas indican gates pendientes.

VEXA — de la carpeta al producto, paso a paso

Esta es la entrada única para construir. El repositorio canónico es ~/vexa; Escritorio/VEXA AI es una copia de entrega sin Git ni estado del loop. No ejecutar el constructor dentro de la copia esperando encontrar los candidatos de la original.

Respuesta exacta: ¿qué automatiza y qué no?

El paquete trae **55 fichas**, un controlador con candidatos y pruebas externas, una ruta interactiva prepare → verify → accept, fixtures monetarios y una guía hasta release. **No es un botón que ya construye todo solo**: hay gates de producto por escribir, dependencias por instalar y permisos/validaciones externas pendientes. Ninguna ficha transforma esos pendientes en PASS.

En esta revisión se substituyó el supuesto irreal de «escribir absolutamente todas las pruebas antes del primer scaffold» por un **ciclo de pruebas externo justo antes de cada incremento**. El control-plane escribe el gate específico de la siguiente ficha, demuestra su rojo, lo revisa/versiona y sólo entonces abre el candidato. El worker jamás escribe su propio examen. Esta es una secuencia de construcción guiada con puntos humanos, no una prueba de autonomía desatendida de meses.

Leer en este orden

1. [Comparación concreta con Likida](#): alcance y brechas, sin una nota arbitraria.
2. [Arranque, comandos y reanudación](#): qué hacer desde una sesión vacía.
3. [Entorno y accesos](#): herramientas verificadas, instalación interactiva, aislamiento y cuentas propias.
4. [Arquitectura y contratos](#): decisiones de implementación, DB/API/roles y ejemplos.
5. [Pruebas y programa del loop](#): quién prepara el examen, fallos, QA, mutación y promoción.
6. [Las 55 fichas en orden](#): un encargo específico por ID, no sólo cinco bullets de fase.
7. [Operación, release y handoff](#): acciones finales y su evidencia.
8. [Rúbrica](#): con qué se evalúa esta guía, no el software futuro.
9. [Evidencia y estado real](#), [revisión inicial](#) y [correcciones](#). El primer veredicto no se oculta.

Versión continua para lectura: [HTML](#) y [PDF](#). Fichas generadas desde scripts/construction_catalog.py + orchestration/graph.json; editar esas fuentes y regenerar antes de abrir candidatos, nunca durante un gate.

Primeros comandos — no instalan ni despliegan

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py readiness
python3 scripts/guide.py next
python3 orchestration/runner.py status
```

- audit comprueba cobertura/paths/entradas/dependencias. Exit 0 **no significa producto listo**.
- readiness sale **2 si faltan gates**. No «arreglar» ese exit borrando tareas.
- next muestra la ficha o la revisión/recuperación pendiente, sin lanzar modelos.
- status es lectura del estado local. accepted acredita el alcance del gate de ese ID, no el SaaS entero.

Qué prueban las suites actuales

npm test corre kernel y preparación/negativos; **no todo el producto**. npm run test:controller prueba el controlador en repos temporales. npm run test:scaffold y test:all-gates siguen rojos mientras falta la app; se conservan, no se desactivaron. Los 47 gates ausentes se escriben bajo revisión en su turno.

Los JSON F00 son snapshots de preparación sintética, no configuración runtime ni una certificación perpetua. Permisos, cuentas y resultados reales posteriores se acreditan en los manifiestos de F07/F08, sin reescribir esos snapshots como si siempre hubieran estado listos.

Tres carriles, una secuencia

Carril	Quién hace el trabajo	Qué puede tocar	Qué nunca acredita solo
Control-plane	Arquitecto/revisor en sesión interactiva	contratos, catálogo, grafo, tests protegidos	calidad de producto por existir un test
Candidato	Codex/Astra u operador en worktree	allowlist de la ficha	editar gate, autoaprobar, acceder a clientes
Externo	Javier y responsables autorizados	cuentas, permisos, etiquetas humanas, piloto, publicación	que un JSON de aprobación sea una firma legal

F00 permite cerrar **preparación sintética** con responsables del piloto pendientes; F07/F08 conservan el candado del piloto/producción. No se inventan contactos ni se reutiliza Supabase de Likida. El orden y los cambios respecto de v2 están documentados en la comparación.

Alcance de treinta días

Días 1–2: fuentes/fixtures/gates/entorno. 2–5: scaffold/Auth/RLS/CI. 4–8: imports/cola. 6–12: conectores. 8–15: IA/evidencia. 12–18: dinero/snapshots. 15–23: producto. 22–27: QA/piloto. 28–30: release/pitch.

Son ventanas que se solapan, no 55 tareas paralelas ni fecha garantizada. Si faltan datos o accesos se puede entregar demo sintética técnicamente validada; no se la llama piloto real. El programa comercial y TAM están en [negocio](#): no cambian por hacer pasar tests.

00 · Comprobación frente al calibre de construcción de Likida

Alcance, sin inflar la comparación

Se leyeron los documentos centrales de construcción, un encargo completo de producto, un encargo completo de QA, el plan de adopción y la parte operativa de la máquina de automejora. También se siguió el enlace al plan de cierre del repositorio canónico ~/Likida, porque el propio índice del blueprint advierte que el original fue superado.

No se auditó toda la compañía Likida ni se revalidó su producción. Sus cifras de pruebas, migraciones, clientes y bugs son afirmaciones de documentos fechados, no mediciones nuevas de esta sesión. No se leyeron archivos de credenciales ni se copiaron datos de clientes/IP. Es comparación de profundidad de los encargos y disciplina de ejecución; no equivalencia entre dos productos ya construidos.

Referencias leídas y rangos

Raíz L: ~/Desktop/Documentos Likida/.

ID	Archivo / líneas	Uso
L1	00-BLUEPRINT-EJECUCION-CLAUDE-CODE.md, 1-338, completo	protocolo, cola, SQL/CI, automejora y merge#deploy
L2	16-Blueprint-de-Construccion/00-PLAN-DE-CIERRE-VIGENTE.md, completo	identifica blueprint superado y fuente vigente
L3	16-Blueprint-de-Construccion/00-PROMPT-SIGUIENTE-SESSION.md, completo	sesión cero-contexto, fuentes, límites, encargos y cierre
L4	16-Blueprint-de-Construccion/fase-1-primer-cliente.md, 1-198, completo	archivo concreto, oráculos y responsabilidad humana
L5	13-Agentes-de-AI/10-Ingenieria-y-Producto/qa-autonomo/encargo-fase-1.md, completo	simulador, escenarios, presupuestos y expected/actual
L6	13-Agentes-de-AI/10-Ingenieria-y-Producto/maquina-de-automejora/plan-de-adopcion.md, completo	secuencia, mutación, propiedades y contraejemplos
L7	misma carpeta, 00-MAQUINA-DE-AUTOMEJORA.md, 1-240 y 460-756	marco y Parte2 operativa completa; NO releída la investigación intermedia 241-459
L8	00-L0-QUE-FALTA-PARA-PUNTA-A-PUNTA.md, completo	fallos vivos, contradicciones de fases y lo humano
R1	~/likida/docs/conocimiento/plan-de-cierre.md, completo	secuenciación posterior; no asumir estado productivo vigente por el título

Metadatos/hasbes del corte en REFERENCIAS-LIKIDA. json. Para citas finas: L1:22-111 protocolo; L1:221-268 automejora; L4:46-118 encargo; L5:51-139 archivos/oráculos; L7:475-756 ciclo/modos/gobernanza/aprendizaje. Los rangos son referencias al archivo leído, no validaciones de sus afirmaciones remotas.

Brechas encontradas de VEXA antes de esta ampliación

1. Fases F00-F08 tenían sólo 5-7 pasos grandes cada una; el grafo los repetía sin suficiente encargo específico.
2. **54 gates ausentes.** El controlador bloqueaba; no había implementación que construyera el SaaS completo por ejecutar run.
3. F00/F08 requerían aprobación, pero sólo había run/status/accept: faltaba preparar/verificar candidato interactivo.
4. Allowlists no cubrían package/lock/CI/fixtures ni ciertos documentos que las tareas pedían entregar.
5. F00 mezclaba derechos/proyectos reales con preparación local; F00-05 pedía escribir todos los gates antes del scaffold, sin un proceso incremental completo.
6. El estado running se podía considerar reintentable sin resolver si seguía vivo; la recuperación estaba insuficientemente guiada.
7. No existía entrada única con prompt de nueva sesión, arquitectura detallada, comandos por tarea, programa de QA y entrega continua en un solo lugar.

Matriz de cierre documental y de herramientas

Dimensión / referencia	Qué exige el calibre	Qué se añade a VEXA	Qué NO demuestra
Fuente vigente L2/R1	no construir una fase obsoleta	entrada única, grafo v3 e histórico v2	que todo el código esté construido
Sesión nueva L3	prompt + estado + próximo paso	01-ARRANQUE, guide next/packet	memoria mágica o loop vivo
Encargo L4	outputs y pasos específicos	55 fichas generadas de catálogo autorado	aceptación de 55 tareas
Dependencias L1/R1	orden que corresponda a realidad	audit DAG/lecturas/allowlists	acceso cloud
Escritura L4/L5	no tocar lo ajeno	14 adiciones puntuales de allowlist, controles protegidos	aislamiento contra malware
Gate L5	oráculo observa efecto	5 gates F00, scaffold/build, kernel compartido; negativos	47 gates futuros escritos
Ruta humana L3	volver al flujo sin falsear estado	prepare/verify/accept, nota explícita	firma legal/autenticación del aprobador
Error L8	conservar incidente/worktree	history, estados, STOP y recuperación supervisada	recuperación perfecta de SIGKILL
Instalación L1	entorno real reproducible	lock/copia temporal/cache offline y preflight	Docker o proyectos ya activos
Datos L1/L5	constraints y aislamiento reales	esquema y matriz Auth/DB/FK/Storage/RPC	RLS ya implementado
Consumer L8	heartbeat y progreso, no cron decorativo	F02-05/06, JOB01..10, alarma	runtime durable alojado probado
Migración CRM	preservar identidad/historia	F03 + alias fixture, comparabilidad	conexión real HubSpot/Zendesk
Finanzas L4/L7	oráculo independiente de LLM	kernel + SYN exacto + mutaciones	ingresos/ahorro causal
IA L5/L7	evaluación fuera del generador	schema/spans/política/holdout/costos	gold humano disponible
UX L1/L7	mirar render y probar efectos	8 rutas, estados y viewport; fichas F06	8 pantallas existentes
Búsqueda L6/L7	seis modos, no sólo unit tests	plan F07 + prueba local de 5 mutaciones	score completo de enterprise
Operación L8	restore, purga y reingesta	06-OPERACION y F07-06	restore de producción probado
Release L1	servido SHA y smoke remoto	F08 manifiesto→smoke→rollback→handoff	despliegue autorizado/hecho
Negocio L4/R1	cliente/pago distinto de demo	checkpoints de piloto y acta por capa	entrevistas/pagos reales
Veredicto L7/L8	evidencia y límites visibles	rúbrica/revisor/comandos/recibos	equivalencia total de toda Likida

Cambio deliberado v2 → v3

IDs y E00 aceptado se preservan. Se archiva el grafo anterior antes de modificarlo, sin regenerar estado ni abrir una corrida activa. Allowlists se amplían sólo para outputs que la ficha necesita. Tareas interactivas/control/externas quedan requires_approval. F00-02/03 distinguen preparación sintética de piloto/proyectos reales; F00-05 pasa a gates externos justo a tiempo. **Eso reduce el bloqueo artificial de desarrollo, no el umbral de piloto/producción.** F07/F08 retienen la validación real.

No importar de Likida instrucciones de autoaplicar migraciones o deploy automático: pertenecen a otro proyecto/otra autorización. No usar sus números de migración, precio o clientes en VEXA. No copiar sus contradicciones: el documento de reanudación y el estado del repositorio mandan sobre un listado antiguo.

Cómo juzgar la equivalencia

Hay equivalencia de **forma de trabajo especificada** cuando una ficha define entradas, efectos, oráculo, comandos, límite y recuperación sin depender de esta conversación. No hay equivalencia de madurez productiva hasta que esas pruebas corran sobre la app y cloud reales. La revisión independiente debe juzgar con la rúbrica del capítulo07, no con «se ve largo» o «tiene más páginas».

01 · Arranque y reanudación sin contexto de conversación

Prompt para pegar a Astra/Codex en una sesión nueva

Trabajas en ~/vexa, no en la copia del Escritorio. Construyes VEXA: inteligencia verificable de posventa, conversaciones-problemas-dinero con evidencia-acciones humanas-medición. Stack producto Next.js/TypeScript, Supabase, Vercel, OpenRouter; para construir usa mi suscripción ChatGPT/Codex, no APIs pagadas.

Lee AGENTS.md, README.md, PROGRESO.md, docs/CONTEXTO-CANONICO.md y construccion/README.md. Ejecuta python3 scripts/guide.py next y runner.py status. No leas audios/transcripciones completos otra vez por falta de contexto; están preservados y el contexto canónico da la interpretación con incertidumbres.

Trabaja SOLO la siguiente ficha del DAG. Si falta gate, estás en control-plane: escríbelo desde los oráculos de esa ficha, prueba el rojo adecuado, pide revisión y congélalo antes de abrir el candidato. No permitas al candidato tocar su prueba. Si está prepared, retoma su worktree; no generes otro. Si está verified, revisa y revalida antes de aceptar. Si running, confirma que no haya proceso vivo.

No actives cloud, datos Senix, conectores reales, envíos o producción por encontrar una llave. No inventes aprobaciones. Registra bloqueo y continúa sólo lo local permitido. Dos fallos requieren diagnóstico y recuperación supervisada. Al cerrar: SHA, archivos, comandos/salidas, expected/observed, no verificado y siguiente paso.

Paso 1 — Comprobar la ubicación y conservar el trabajo

```
cd ~/vexa
pwd
git status --short
git log -3 --oneline
git worktree list
python3 orchestration/runner.py status
python3 scripts/guide.py next
```

Si hay cambios sin commit, **no hacer reset ni clean**. Clasificarlos: propios, otro agente, usuario. Leer diff y decidir integración por separado. Si el checkout es copia de entrega sin `.git`, volver al canónico; no inicializar un segundo historial sobre la copia para simular continuación.

Paso 2 — Convertir la ficha en un examen externo

Ejemplo (ID de ejemplo, usar el que next indique):

```
python3 scripts/guide.py packet --task F02-05
```

1. Leer contrato de job/lease en 03-CONTRATOS.md y los oráculos específicos.
2. Escribir tests/acceptance/F02-05.test.mjs **en control-plane**. Debe apuntar al código de VEXA_CANDIDATE, no importar el baseline por accidente.
3. Preparar DB/worker de ensayo según el capítulo 04. Hacer fallar la prueba suprimiendo fence, y comprobar que el fallo es una publicación doble/obsoleta, no «no encontré psql».
4. Congelar expected/fixture/semilla, revisión y gate; actualizar registro y generar fichas. Hacer commit local revisado. No mover HEAD mientras un candidato está preparado.
5. Si el gate requiere más de cinco minutos, separar el incremento o usar carril de QA supervisado. No agrandar límites del constructor para tapar una tarea mal partida.

El grafo v3 **no autoriza al worker a crear gates**. La antigua tarea F00-05 que exigía gates de todo el SaaS antes del scaffold fue reemplazada por este protocolo justo a tiempo. El histórico v2 sigue en orchestration/history/.

Paso 3 — Candidato interactivo (ruta universal)

```
python3 orchestration/runner.py prepare --task F00-01 --max-minutes 1
```

prepare imprime ruta de worktree y persiste prepared, sin llamar un modelo, sin copiar privados/credenciales, sin aceptar. Obtener la ruta del recibo:

```
export TASK=F00-01
export CANDIDATE="$(python3 -c 'import json,os; s=json.load(open(\".runtime/state.json\")); print(s[os.environ[\"TASK\"]][\"worktree\"])\")"
printf '%s\n' "$CANDIDATE"
git -C "$CANDIDATE" status --short
```

Editar únicamente ese worktree y su allowlist; no hacer commit allí. Se pueden repartir las ediciones en sesiones acotadas manteniendo el mismo prepared. El gate se corre desde el control-plane, no desde una prueba que el candidato se haya escrito a sí mismo.

```
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F00-01.test.mjs
python3 orchestration/runner.py verify --task F00-01 --max-minutes 5 \
  --approval-note 'Referencia real a la decisión del operador sobre preparación sintética'
```

No copiar la nota como si fuera una aprobación real. `--approval-note` registra una atestación textual, no autentica a una persona ni verifica un contrato. Para una tarea externa hay que hacer el acto y conservar evidencia antes de verificarla. El texto no concede permisos que no existan.

Si pasa, queda verified; nunca merge automático por este camino. Revisar `git diff <baseline> <candidate_commit>`, logs y expected/observed. Después:

```
python3 orchestration/runner.py accept --task F00-01 --max-minutes 5
python3 scripts/guide.py next
```

accept vuelve a ejecutar la prueba y comprueba firmas/HEAD antes del fast-forward. Cambiar contrato/gate/candidato después de preparar invalida el recibo. Si se rechaza un candidato verified, usar reject como se indica abajo; si está failed/blocked, recover. Nunca forzar el merge.

Paso 4 — Worker offline, sólo con gate y entorno listos

```
codex login status
python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20
```

Debe decir Logged in using ChatGPT. El worker usa gpt-6-astra, sandbox workspace-write y entorno sin API keys heredadas. No instala paquetes por red ni publica. Las tareas requires_approval van por la ruta interactiva. run selecciona la próxima elegible: no usar --task como selector de ejecución automática; ese selector es para prepare/verify/accept.

Paso 5 — Qué hacer con cada estado

Estado	Acción siguiente	Qué no hacer
pending + gate missing	Escribir examen protegido desde ficha, revisarlo y versionar	empezar worker sin examen
pending + deps pendientes	Terminar/aceptar dependencia	editar dependencias para saltarla
prepared	Retomar worktree registrado, luego verify	otro prepare encima
verified	Revisión limpia; accept revalida, o STOP + reject si se rechaza	editar y forzar merge
failed, intento 1	Leer log, reproducción mínima; nuevo prepare conserva history; trasladar sólo patch permitido	borrar intento malo
failed, 2 intentos	Escalada y recuperación revisada	poner attempts=0 automáticamente
blocked por guard	Congelar evidencia, reparar causa fuera del candidato	quitar guard o modificar status
running huérfano	Inspeccionar PID/grupo/worktree; detener sólo proceso identificado y autorizado	relanzar a ciegas
accepted	Siguiente dependencia elegible	reejecutar fase ya aceptada para producir commits

Recuperación supervisada de failed/blocked: primero preservar evidencia, confirmar que no hay procesos vivos y corregir la causa en control-plane. Con checkout limpio, gate revisado y dependencias aceptadas:

```
touch .runtime/STOP
python3 orchestration/runner.py recover --task F00-01 \
  --approval-note 'REFERENCIA REAL: causa revisada y nuevo ciclo acotado autorizado'
# Revisar history: recover deja pending/attempts=0 y conserva intentos/worktrees.
# Sólo tras revisión del operador:
rm .runtime/STOP
```

Esto abre explícitamente un presupuesto NUEVO de dos intentos y registra recovery_count; no es un reintento infinito automático. No sirve para accepted/verified/prepared/running ni acepta nada. Running huérfano sigue necesitando investigación y recuperación del control revisada; no se transforma ciegamente a failed. No hay auto-rebase. La nota de ejemplo no es autorización real.

Rechazo después de verify

Si el revisor pide corregir un candidato verified (incluso si ya se editó por error o falla el recheck), **no aceptarlo ni falsear estado:**

```
touch .runtime/STOP
python3 orchestration/runner.py reject --task F00-01 \
  --approval-note 'REFERENCIA REAL: revisión rechazada, causa y nuevo ciclo autorizado'
# Revisar history, commit/worktree original y logs conservados.
# Tras autorización, retirar STOP y preparar un candidato NUEVO.
rm .runtime/STOP
python3 orchestration/runner.py prepare --task F00-01 --max-minutes 1
```

reject sólo admite verified; no reabre accepted/prepared/running. Archiva el recibo anterior y abre pending con dos intentos explícitamente nuevos. Llevar manualmente sólo la corrección permitida al nuevo worktree y repetir verify/revisión/accept. No hace auto-rebase ni borra el candidato rechazado. Un recibo verified antiguo sin firma completa también requiere este camino.

Los logs de prepare, worker, gate, verify y cada recheck tienen ID único. El recibo guarda nombre y SHA256 al cerrar esa ejecución; history conserva las referencias. Recuperar o repetir un recheck no sobrescribe la salida anterior.

Pausa / cierre

```
mkdir -p .runtime
touch .runtime/STOP
```

Impide siguientes vueltas; no interrumpe instantáneamente la actual. Ctrl-C del controlador intenta terminar el grupo del worker. Procesos que se desligan del grupo requieren inspección del host. Retirar STOP sólo después de revisar recibos/procesos; conservar motivo en handoff. No dejar un bucle indefinido abierto en Terminal.

02 · Entorno, instalación y accesos sin mezclar cuentas

Lo observado en esta Mac durante esta ampliación

node --version → v26.7.0; Python 3.9.6; Git 2.54.0; Codex CLI 0.154.0; Supabase CLI 2.115.0; Vercel CLI 59.1.4; Docker CLI 29.7.2. codex login status confirmó ChatGPT. **CLI instalado no prueba Docker daemon activo, proyectos cloud, permisos ni presupuesto.** No se actualizaron estas herramientas como efecto secundario de escribir la guía.

El mínimo del kernel/controlador es Node>=22 y Python3. Para el SaaS fijar versión de Node compatible con Next y Vercel en F01-01; no inferir compatibilidad por la versión instalada en esta Mac. Las dependencias del informe son separadas: requirements-negocio.txt.

Preparación local — operador, antes del worker offline

1. Reservar repo y directorio VEXA; no tomar .env de Likida/Atiende ni sus project refs.
2. Confirmar git status, codex login status, Node/Python y espacio libre.
3. Comprobar Docker: docker info. Si no funciona, iniciar Docker Desktop con autorización o declarar DB integration blocked. No confundir docker --version con daemon sano.
4. Obtener versiones desde registro público en sesión interactiva, no en worker: bash npm view next version npm view react version npm view typescript version npm view create-next-app version Guardar fecha/versiones/resolución en lock. Es consulta futura para F01-01; no se declara ejecutada en esta preparación.
5. Ejecutar create-next-app con versión explícita inspeccionando --help de ESA versión: App Router, TypeScript, src/, npm, directorio apps/web. Preservar package.json y scripts existentes, añadir npm workspaces. No usar Next como API gateway de OpenRouter sin autorización/tenancy.
6. Instalar con npm ci --ignore-scripts en **copia temporal revisada** para llenar cache. Inspeccionar cualquier script de instalación necesario antes de permitirlo. Nunca ejecutar instaladores sobre otro repo para «ahorrar dependencias».
7. El gate F01-01 copia inputs a directorio temporal, hace npm ci --offline --ignore-scripts, lint/typecheck/build y borra SOLO ese temporal propio. No crea .next/node_modules dentro del candidato: el controlador interactivo rechaza archivos ignorados/mutación durante pruebas. Cache incompleta es bloqueo de entorno, no fallo de negocio ni mutante muerto.
8. Preparar Supabase local con supabase init y supabase start en el proyecto aislado después de revisar la configuración. Aplicar migraciones exclusivamente contra esa instancia. supabase db reset --local es destructivo para la DB local: sólo namespace de pruebas desechable confirmado, nunca producción.

No hay que gastar OpenRouter para probar Auth/SQL/imports/tenancy. CRM y LLM comienzan como stubs deterministas que se rotulan; **DB/Auth/Storage de ensayo sí deben ser reales** cuando el criterio los requiere.

Separación de variables

Categoría	Ejemplos de nombres	Dónde se guarda	Nunca
Públicas del cliente	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY	config pública de entorno autorizado	service_role o OpenRouter en NEXT_PUBLIC
Servidor	SUPABASE_SERVICE_ROLE_KEY, OPENROUTER_API_KEY, CRM_CLIENT_SECRET	secret store del entorno	commit, browser, captura, prompt de worker
Desarrollo/fixture	URL/credenciales de instancia local efímera	temporal del verificador fuera del candidato	reutilizar en otro tenant/producción
Constructor	OAuth de Codex/ChatGPT	almacén propio de Codex	conversión a API-key fallback

El controlador limpia variables del worker y su gate; no basta hacer export de una llave esperando que se herede. Para integración, el **verificador externo** aprovisiona contexto efímero local y pasa sólo esos valores al proceso de ensayo. El candidato no decide host/proyecto ni obtiene llaves reales. No agregar SUPABASE_SERVICE_ROLE_KEY a clean_environment para quitar un bloqueo.

Derechos/cuentas: quién desbloquea qué

Falta	Quién responde	Evidencia mínima	Qué se puede hacer mientras
Roles sponsor/CRM/privacidad/finanzas	interlocutor autorizado del piloto	nombre, rol, aceptación, referencia	fichas/fixtures locales
Datos Senix/YAT	titular/controlador y privacidad	finalidad, campos, subprocesadores, retención, permiso	sólo sintético
HubSpot	admin cuenta propia del piloto	app/scopes/endpoint de mensajes y prueba read-only autorizada	contrato con stub
Zendesk	admin de instancia	subdominio/scopes/comments/cursor confirmados	contrato con stub
Vercel/Supabase/GitHub VEXA	Javier/owner designado	project refs, entornos, presupuesto, permiso específico	scaffold y CI local
OpenRouter real	owner presupuesto/privacidad	política endpoint/modelo/retención/región y tope	stub sin costo proveedor
Logo/caso/pitch identificable	titular de marca/datos	autorización de publicación, no sólo NDA	marca propia + demo

Una aprobación puede ser limitada a un entorno/fixture, no a todos los pasos futuros. Estados de F00 significan preparación local; no rebautizarlos como integración o piloto terminado.

Cuándo el entorno merece más aislamiento

Un Git worktree comparte host y .git. Las firmas detectan cambios, no impiden leer secretos de HOME. Con código/paquetes no confiables, usar VM/contenedor desechable sin llaves, sin mounts de otros repos y red denegada; no montar el HOME real por comodidad. Esta entrega prueba un controlador para agentes cooperativos con errores, no resistencia a malware.

03 · Arquitectura y contratos de implementación

Especificación normativa propuesta para construir, no inventario de tablas/endpoints existentes. Precedencia: requisito explícito del usuario → contexto canónico → contratos financieros/seguridad → este detalle → ficha → código comprobado. Una contradicción se registra y resuelve en control-plane; no se arregla cambiando tests dentro del candidato.

ADR-01 · Una aplicación, módulos de dominio

npm workspaces; Next.js App Router TypeScript en apps/web, módulos en packages/{platform, ingestion, jobs, connectors, gateway, intelligence, problems, economics, metrics, recommendations, interventions, briefs}. Supabase Auth/Postgres/Storage; pgvector con modelo/dim/version. APIs de la app llaman servicios de dominio; esos servicios llaman puertos Clock, IdGenerator, SourceTransport, ModelTransport, Repository. Interfaces inyectables para probar errores sin cambiar producción.

No crear ocho microservicios, MCP server obligatorio ni agentes de negocio autónomos. OpenRouter es transporte del producto; Codex OAuth sólo construcción. Dinero permanece en kernel determinista existente. La capa HTTP/UI no contiene una segunda fórmula monetaria.

ADR-02 · Trabajo durable antes que framework de workflow

Base seleccionada para implementación local: tablas outbox/jobs/checkpoints en Postgres, consumidor separado del request de carga, transacciones/fencing e idempotencia. Se puede demostrar localmente sin comprar otro servicio.

Transporte alojado pendiente de spike F02-05/06: evaluar Supabase Queues y activación por Vercel de chunks cortos; no sumar Vercel Workflow además por defecto. Para elegir, ejecutar el mismo fixture con crash después del commit, lease expirado, reintento, backlog y consumidor apagado. Registrar límites efectivos del plan y latencia bajo carga. Si el consumidor no progresa dentro del presupuesto objetivo, bloquear preview hasta escoger/autorizar runtime durable adecuado. La guía no afirma que un cron de un minuto cumpla cualquier SLA.

El transporte puede variar sin cambiar contrato de efectos/checkpoints. Procesamiento at-least-once: no prometer exactly-once extremo a extremo por una descripción comercial de cola.

ADR-03 · Esquema, identidad y permisos

IDs internos UUID; IDs remotos string. Toda entidad privada tiene tenant_id uuid NOT NULL, id, timestamps y procedencia. Unique (tenant_id, id) habilita FK compuesta. Registros globales sólo para catálogos públicos. ON DELETE explícito: no anular tenant_id con un SET NULL general.

Grupo / tablas	Columnas esenciales además de tenant/id	Índices/reglas críticas
organizations, memberships	memberships: user_id, role, status, permissions, version	unique tenant/user; consulta membership vigente
connections	source, account_id, status, credential_ref, watermark, last_success	unique tenant/source/account; nunca token en JSON cliente
external_aliases	source/account/entity/external_id, canonical_id, evidence_ref, approved_by, version	unique identidad por versión; mismo tenant en ambos lados
imports/import_rows	file_hash, mapping_version, state, total/accepted/rejected/duplicates/pending; row_ref/error	unique tenant/idempotency_key; suma de contadores
customers/products/orders/order_lines	external refs, customer_id?, SKU?, amount_minor?, currency, exponent, occurred_at	FK tenant-aware; moneda y exponente no implícitos
conversations/messages/message_revisions	source identity, revision, role, text_ref, hash, occurred_at, deleted_at	unique tenant/connection/entity/external_id/revision
jobs/attempts/checkpoints/outbox/dead_letters	state, input_ref, lease_until, fencing_token, next_attempt_at, deadline, checkpoint	tenant/type/input_hash/version unique; index state/next_attempt
extraction_runs/issues/evidence_spans	model/prompt/schema hash, status, usage; message_revision_id, start, end, quote_hash	ownership y revisión, offsets code points
embeddings/problems/problem_versions/problem_conversations	model/dim/version; stable problem_id, severity, cause_status, members	retrieval tenant-filtered; split/merge auditados
economic_events/reversals/cost_rates/assumptions	kind, status, amount_minor, currency, effective_at, source_ref, reversal_of	dedup de evento; reversal misma moneda/original
metric_snapshots/components/attributions	scope_hash, input_hash, policy_version, watermark, status, bundle_ref	publicar atómicamente; snapshot publicado inmutable
recommendations/interventions/measurement_plans/weekly_briefs	evidence_refs, owner, status, baseline_ref, measurement_ref, version	optimistic concurrency + idempotency; acciones humanas
audit_events/tombstones	actor, action, resource, reason, trace_id; deleted_source_key	no PII cruda; restore/reingesta respetan borrados

Secuencia SQL prevista: 0001 identidad, 0002 fuentes, 0003 ejecución, 0004 evidencia/dinero; 0005+ extensiones por ficha. Estos números no están reservados globalmente: antes de escribir, listar migraciones actuales y asignar siguiente libre. No reutilizar un número aplicado. Cada archivo debe poder aplicarse en una DB local vacía en orden y la suite debe detectar FK/RLS/uniqueness incorrectas.

Matriz RBAC

Acción	owner	analyst	operator	viewer
Leer snapshots/evidencia/export autorizado	sí	sí	sí	sí
Importar/configurar mapping	sí	sí	no	no
Conectar fuente/cambiar política/miembros	sí	no	no	no
Proponer recomendación/intervención	sí	sí	sí	no
Aprobar intervención/política	sí	no	no	no
Registrar ejecución/medición asignada	sí	no	sí	no

Acción	owner	analyst	operator	viewer
Borrado/retención/costos	sí, con confirmación	no	no	no

admin histórico significa owner, no rol nuevo. El tenant se deriva de identidad + membership vigente; selector de org sólo escoge entre memberships, jamás concede uno. Sesión autenticada sin membership no accede. RLS/Storage/RPC no dependen del sidebar. Acceso cross-tenant devuelve404 en recursos privados; identidad ausente401, rol insuficiente403; conflicto de estado409.

ADR-04 · API del producto (a implementar)

Prefijo /api; contract_version en respuestas. JSON uniforme: {data,meta:{trace_id,snapshot_id?,scope_hash?,coverage?,state?}}; errores {error:{code,message,retryable},meta:{trace_id}}, sin stack/token/PII. No devolver200conarrayvacío ante DB caída; usar503 y UI error/stale.

Método/ruta	Input / efecto	Rechazos obligatorios
POST /api/imports	file metadata + idempotency key; reserva import/upload scoped	rol, tipo, bytes, cuota
POST /api/imports/:id/confirm	objeto/hash/mapping_version; transacción import+outbox	objeto ajeno, token expirado, hash distinto
GET /api/imports/:id	preview/mapping/row errors y contadores	tenant ajeno404
GET /api/jobs/:id	queued/running/partial/succeeded/failed/cancelled	no éxito por202
POST /api/jobs/:id/cancel	solicita cancelación idempotente	rol/estado inválido
GET /api/metrics	scope validado, mismo bundle que UI	fechas/moneda/basis inválidos400
GET /api/problems y /:id	paginación estable; cifra→eventos→citas	cursor de otro scope, revisión no autorizada
GET /api/customers/:id	historial por identidad conocida	null no crea cliente ficticio
GET /api/connections	health/cobertura/reconnect	no credenciales en respuesta
POST /api/recommendations/:id/interventions	draft + idempotency key	viewer403, evidencia faltante
POST /api/interventions/:id/transition	expected_version,target,reason,measurement_ref	transición/versión inválida409
POST /api/briefs	snapshot/scope/version	cifras no respaldadas, scope mismatch
POST /api/exports	snapshot/scope, formato	parcial sin modo explícito
GET /api/exports/:id/download	acceso revalidado; descarga scoped	membership revocada, expiración
POST /api/explorer/query	pregunta + scope; tools read-only	SQL libre, herramienta no allowlisted
GET /api/health/version	SHA/config hash no secretos	no estado sensible de todos los tenants

Las rutas de login/callback dependen de la integración SSR documentada de la versión elegida. No inventar tokens válidos con un JSON base64 en tests; usar Auth real local para probar sesión.

Scope y dinero: ejemplos exactos

```
{
  "date_start": "2026-09-01T00:00:00Z",
  "date_end": "2026-10-01T00:00:00Z",
  "timezone": "UTC",
  "date_basis": "order",
  "currency": "USD",
  "sku": [],
  "source": [],
  "snapshot_id": "synthetic-S1"
}
```

tenant_id se añade en servidor desde sesión. scope_hash se calcula sobre serialización canónica incluyendo permisos/versiones, filtros y snapshot; no confiar en hash enviado por browser. Cursor se liga a ese scope y sort con desempate ID. Agregados SQL sobre TODA la población, no sobre la primera página.

```
{
  "metric": "revenue_exposure",
  "amount_minor": "30000",
  "currency": "USD",
  "exponent": 2,
  "kind": "observed-order-exposure-not-loss",
  "additive_across_problems": false,
  "order_ids": ["01", "02"],
  "scope_hash": "calculado-no-literal",
  "snapshot_id": "synthetic-S1"
}
```

Si falta O3.amount: total todas órdenes=null, known_subtotal=30000, known_count=2,total_count=3,partial. P1=30000 y P2=10000 NO hacen40000 global. Refund neto=1500; replacement=1200 observado; soporte=500 modelado. No se suman como «VEXA recuperó3200». USD/MXN separados salvo FX aprobado/versionado. Fuente financiera es ledger/orden, no frase del ticket.

ADR-05 · IA con citas, abstención y costo

extract(redactedMessages,taxonomy,policy) devuelve issue/sentiment/intent/urgency/entities/evidence/abstention. No importes contables libres. Cada evidence_span referencia message_revision autorizada y offsets en **Unicode code points** sobre texto redactado versionado. Emoji no debe partirse como bytes/UTF16 indistintamente. Nota interna no se etiqueta como voz del cliente.

Gateway reserva antes de red, concilia usage y conserva reserva uncertain ante timeout. Fallback debe satisfacer mismos requisitos de proveedor/retención/residencia. No endpoint elegible → policy_blocked. data_collection:deny no prueba ZDR. El mock sólo verifica manejo del contrato; precisión requiere gold humano temporal y WTP requiere entrevistas.

ADR-06 · Ocho vistas, no ocho nombres vacíos

Overview, Problems, Problem detail, Recommendations, Explorer, Customer, Interventions, Brief. Las fichas F06-01 y F06-02 cubren dos rutas cada una; las restantes cubren acciones, export y accesibilidad. Cada vista debe probar loading/empty/error/partial/stale/ready; UI/API/export comparten snapshot. Las aserciones exactas están en docs/blueprint/calidad/04-aceptacion-end-to-end.md y fixture SYN-E2E-v1.

ADR-07 · Intervención y causalidad

Draft→approved(owner)→active(owner/operator asignado)→measuring→closed; cancelled con razón/auditoría. Baseline/hipótesis/unidad/población/fechas/plan se fijan antes de ejecutar. Cierre sin medición409. Antes/después es asociación, no ahorro causal; cambios de temporada, cobertura, mix de clientes y migración deben tratarse. Ninguna transición modifica CRM o emite un refund por sí sola.

04 · Programa de construcción, examen externo y automejora

Dos loops distintos

Construcción: requisito → gate → candidato → pruebas → revisión → aceptación → siguiente tarea. Es el controlador local de `orchestration/runner.py`.

Calidad del producto: generar escenario → ejecutar app/DB/worker → comparar oráculos → reproducir bug → regresión roja → fix → revalidar → documentar clase de fallo. Se implementa progresivamente en F07 sobre la app, no se declara existente por tener un controlador de desarrollo.

No agregar agentes de negocio al SaaS porque el constructor use agentes. No convertir un loop de ingeniería en permiso de CRM, correo, refunds o publicación.

Programa del loop (reglas de trinquete)

- **Métrica:** criterios del incremento que pasan por verificación externa y revisión; dirección creciente. N° de documentos, tokens y commits no es progreso.
- **Guardias:** cero regresión financiera/aislamiento; controles y expected intactos; sin datos/acciones no autorizados.
- **KEEP:** gate0, firmas/HEAD intactos, diff permitido, revisión; accept revalida antes de fast-forward.
- **REVERT:** el baseline nunca recibe candidato fallido. Conservar `worktree/log` como evidencia; no reset/clean de trabajo ajeno.
- **Presupuesto:** una vuelta por defecto, `workers`≤900s, `gates`≤300s, dos intentos por tarea; tiempo de sesión explícito. Instalaciones/red/QA larga son interactivas fuera del worker.
- **STOP:** gate ausente, dependencia pendiente, aprobación, 2 fallos, guardia, STOP o presupuesto. No renombrar blocked a pass.
- **Crash:** prepared se retoma; running huérfano necesita inspección. No prometer recuperación perfecta de SIGKILL/procesos desligados. Trabajo largo puede hacerse por varias sesiones en un prepared, sin aceptar a medias.
- **Aprendizaje:** reproducción y regresión permanente; dos ocurrencias de una clase justifican regla estructural; un snapshot actualizado no reemplaza una explicación de por qué cambió.

Estado medible de gates

```
python3 scripts/guide.py readiness
```

El inventario inicial de esta ampliación tiene **8 gates de tareas presentes y 47 pendientes**. Los de F00 comprueban preparación/fixtures, no software. F01-01 es el gate de scaffold/build y **falla mientras no exista el scaffold**. F05-01 reutiliza el kernel, no prueba SQL/UI. Un gate pendiente tiene oráculos específicos en su ficha; se escribe y revisa justo antes de ese incremento.

Esto evita dos trampas: exigir tests de toda una app desconocida antes de empezar, o crear 54 archivos que sólo comprueban que existe un Markdown. **No se anuncia que todos los gates estén escritos**. Si el usuario quiere una corrida totalmente desatendida, este estado no la acredita.

Cómo escribir un gate de función pura

Ejemplo orientativo para F04-04; la función es una interfaz a implementar, no código ya existente:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import path from 'node:path';
import {pathToFileURL} from 'node:url';
const candidate = process.env.VEXA_CANDIDATE;
assert.ok(candidate, 'candidate requerido; no importar baseline por accidente');
// Compilar TS a un temporal, o usar el loader fijado en F01; nunca inventar dist.
const {validateEvidence} = await import(pathToFileURL(path.join(candidate, 'packages/intelligence/evidence.mjs')));
test('rechaza cita del tenant B incluso si el texto coincide', () => {
  assert.throws(() => validateEvidence(
    {tenant:'A', revisionId:'B-r1', start:0, end:4, quote:'hola'},
    [{tenant:'B', id:'B-r1', text:'hola', role:'customer'}]
  ));
});
```

Ajustar la importación al módulo **real** creado por el contrato/ficha (TypeScript `src/evidence.ts` compilado en temporal). No crear una fachada que pase el test pero que la app no use. Añadir prueba de ruta HTTP→servicio→DB en F06/F07 para acreditar cableado. El ejemplo ilustra el oráculo, no es un archivo de gate entregado.

Gates de integración: no se sustituyen por mocks

DB/RLS

1. Verificador externo crea Supabase/Postgres efímero de ensayo, no instancia del cliente.
2. Aplica migraciones del candidato en orden. Sólo el setup usa servicio privilegiado.
3. Crea A/B y usuarios mediante Auth real local; inserta SYN-E2E-v1 con identidades distintas y canario B.
4. Ejecuta SELECT/INSERT/UPDATE/DELETE/FK/RPC/Storage como anon y usuarios A/B, roles de matriz; afirmar por valor y códigos exactos, no «ocurrió algún error».
5. Revoca membership entre requests y prueba cache/export/jobs. Consultar DB directamente para verificar persistencia.
6. Teardown sólo instancia/namespace propiedad de esa corrida. Si no pudo levantarse, blocked, nunca skip verde.

Consumidor y durabilidad

Request→import/outbox atómicos→claim→efectos/checkpoint→ack→snapshot→lectura. Usar barreras explícitas para pausar entre commit y ack y para vencer lease antes del publish. Dos procesos reales sobre la misma DB; fake timers de Node no controlan reloj del servidor Postgres. Inyectar now en lógica y establecer las condiciones de lease en DB de ensayo bajo control del test.

Comparar conjuntos de IDs, no sólo COUNT(*): se puede perder una fila y duplicar otra conservando el conteo. Verificar oldest queued y alarma al apagar consumidor. Reordenar/repetir páginas/cancelar deben preservar garantías.

Browser/HTTP

Playwright sobre la app **propia**. Usuarios A_owner/A_analyst/A_operator/A_viewer/B_owner. Abrir ocho rutas, usar botones reales, comprobar response y estado DB; el screenshot por sí solo no acredita POST exitoso. Medir a 390x844 y 1440x900, teclado/reduced-motion. Descargar export y comparar su scope/cifras con API/snapshot, no imagen parecida.

Modelos/CRMs

Mocks sirven para 429/401/JSON inválido/reintentos/cursores y citas. No sirven para confirmar rutas/scopes efectivas ni precisión. Live tests necesitan permiso, presupuesto, datos mínimos y logs redactados. Anotar proveedor/endpoint/modelo/política y versión. Un catálogo de modelos accesible no implica elegibilidad contractual.

Fixture exacto, bancos separados

- tests/fixtures/syn-e2e-v1.json: A/B, alias, 4 conversaciones A, 3 órdenes A, exposición global30000 USD, refund neto1500, replacement1200, soporte modelado500.
- SYN-PAGE-v1: 101 conversaciones neutrales distintas; paginación sin huecos/duplicados. No mezclarlas con totales del smoke.
- SYN-LOAD-v1: datasets10K/50K/150K con manifest por corrida y semilla. No decir que la precisión está validada por usar 150K textos repetidos.
- Gold humano: corpus privado consentido y versionado, train/dev/holdout separado por cliente/tiempo. Sin humano es synthetic, aunque el modelo «etiquete» muy bien.

Matriz de promoción por capa

Capa	Casos mínimos	Oráculo independiente	Punto de promoción
Dinero	FIN01..12 + replay/null/moneda/reversal/BigInt	kernel y expected de fixture	F05, sin modificar expected
Aislamiento	API/DB/FK/Storage/vector/export/cache/jobs	canario B y consultas como usuario	F01/F07, ninguna fuga
Ingesta	corruptos//límites/mapping/quarantine	filas originales vs aceptadas/rechazadas/duplicadas/pendientes	F02
Durabilidad	JOB01..10	efectos/checkpoints/fence en DB	F02/F07
Migración	alias/revisión/deletes/cobertura	conjuntos antes/después, hashes de scope	F03
IA	schema/quote/role/inyección/PII/política	revisiones autorizadas y gold externo	F04
UX	UI01..08 + NEG01..08	DOM+HTTP+DB+export; revisión visual	F06
Operación	restore/rollback/tombstone/revoke/alarma	hashes/conteos y ensayo en entorno nuevo	F07/F08
Comercial	comprensión/WTP/insight/acción	personas autorizadas y datos observados	piloto, no unit tests

IDs y pasos ampliados en los cinco dossiers de docs/blueprint/calidad/; no se eliminan para mejorar un score agregado.

Los seis modos de búsqueda adaptados a VEXA

- Propiedades:** montos enteros arbitrarios, cero/null, extremos, dedup; kernel ya tiene generación con semilla.
- Metamórficas:** orden/replay/partición de lotes/alias no cambian valor si no cambió identidad económica.
- Mutación:** quitar dedup, convertir null a0, sumar reversal, perder precisión, inflar escenario. En esta ampliación scripts/probe_economics_mutations.py prueba cinco mutaciones concretas en temporales. Resultado guardado en RESULTADO-MUTACION.json; **no es un mutation score completo del SaaS**.
- Fuzzing:** CSV/XLSX corruptos, zip bomb, fórmulas, encodings, Unicode y prompts hostiles; ningún ataque toca cuentas de terceros.
- Carreras/caos:** two claims, lease viejo, retry after commit, DB error por valor, timeout con costo incierto, revocación en vuelo.
- Red team con objetivo:** «extrae SOLO_B_9F como A» o «haz que UI y export discrepen con mismo scope». Componer pasos válidos, no sólo inputs absurdos.

Un mutante sólo es killed si falló una aserción relevante; syntax error/timeout/setup fallido se registran aparte. No subir umbral a100% con cinco mutaciones seleccionadas. Cada hallazgo debe reproducirse, quedar con seed/archivo/línea y conservarse en regresión.

Qué se aprende de Likida sin copiar sus errores

Los documentos de Likida distinguen cobertura y protección, relatan crones sin heartbeat, lectores inexistentes y drift merge/deploy. VEXA adopta esa exigencia de observación. No importa sus autorizaciones de migrar automáticamente ni sus porcentajes históricos. «Nunca tocar pruebas» aplica al candidato; el control-plane sí mejora pruebas mediante revisión y rojo real, o el sistema no podría aprender.

06 · Operación, release y entrega de punta a punta

Runbooks para implementar y ensayar en F07/F08. No afirman que esos sistemas estén operando hoy. Producción, datos reales y publicación necesitan autorización específica, además de tests verdes.

Runbook A — Cola detenida o trabajador caído

- Owner técnico confirma entorno/tenant/job_id; mirar last_success, heartbeat, checkpoint, oldest_queued y lease. No imprimir payload/credenciales.
- Pausar admisión de nuevos imports si el consumidor no progresa. Mantener UI queued/stale con motivo; no marcar succeeded por un202.
- Consultar intents/outbox y efectos ya comprometidos. Identificar si murió antes o después del commit. No incrementar cursor manualmente.
- Reparar consumidor/acceso y reanudar desde checkpoint con nuevo fencing token; trabajador anterior no debe publicar.
- Repetir fixture de fallo con oráculos de conjunto de IDs, ledger y snapshot únicos. Confirmar que antigüedad de cola baja y que UI deja de mostrar alarma por progreso real.
- Registrar causa, tiempo observado de recuperación, intento, versión y prueba permanente. Si el fallo se repite, regla estructural/alarma, no sólo postmortem.

Runbook B — Credencial revocada o gasto incierto

- 401/403 → connection reconnect_required, detener esa sincronización; no insistir cuatro veces por reflejo.
- Token queda fuera de logs y de la UI. Owner autorizado reconecta por el mecanismo revisado; scopes mínimos read-only.
- Reanudar desde último cursor comprometido y dedup; no volver a cargar todo para «empezar limpio».
- Para timeout LLM: reserva uncertain y request_ref redactado. Consultar registro de uso autorizado para conciliar; no costo0 ni liberación por falta de respuesta.
- Si no se puede conciliar, mantener incertidumbre y presupuesto retenido con intervención humana. La suscripción de Codex no paga ese saldo.

Runbook C — Eliminación y restore

- Verificar identidad/permiso de la solicitud y alcance/retención legal; crear tombstone antes de permitir reingesta.
- Borrar o anonimizar los derivados que la política promete: raw, texto, embeddings, caches, exports y referencias. Conservar sólo auditoría mínima autorizada.
- Backups: explicar ventana/expiración y procedimiento de reaplicar tombstones al restaurar. No prometer eliminación instantánea de un backup que no puede editarse.
- Ensayo en instancia NUEVA local: restaurar snapshot/backup, reaplicar tombstones, comparar conteos/hasheos y verificar que contenido borrado no reaparece ni por búsqueda semántica.
- Medir RPO/RTO del ensayo con hora/entorno/volumen. No decir «backup listo» sólo porque el archivo existe.
- Prohibido probar reset/destrucción contra producción. Un fallo de restore bloquea release.

Runbook D — Release autorizado

Antes del primer push

- Confirmar nombre del repositorio VEXA, propietario y visibilidad. El canónico no debe inferir un remote de otro proyecto.
- Revisar git diff, archivos staged, secrets y licencias; private/, .runtime/, .env*, .vercel/ y credenciales nunca se publican. La carpeta local contiene fuentes privadas: no subirla entera.
- Crear proyecto Supabase/Vercel exclusivo de VEXA, entorno preview separado y presupuesto aprobado. Instalar secretos en secret store correspondiente, no en una ficha.
- Revisar workflow de CI; sin permisos de fork privilegiado, sin skip de pruebas críticas, lock congelado. SHA del CI = SHA que se propone desplegar.

Orden de promoción (operador con permisos)

- Run unit/kernel, Auth/RLS/Storage/DB, integración, browser, mutación/casos críticos; todo contra el SHA exacto.
- Generar docs/entrega/release-manifest.json: SHA, lock/config hashes, migrations, target, owners, run_ids, backup/restore y bloqueos.
- Revisar migraciones con estrategia expand/contract y compatibilidad del SHA anterior. Enumerar/aplicar archivos en orden; no editar migraciones ya aplicadas.
- Desplegar preview desde root directory configurado apps/web, después de comprobar proyecto. Usar el CLI instalado y sus flags comprobados con vercel deploy --help; el permiso de preview no autoriza producción.
- Leer /api/health/version en URL efectiva y comparar SHA con manifest; deployment READY no sustituye smoke.
- Ejecutar SYN-E2E-v1 desde browser remoto: login→upload→job terminal real→ocho rutas→acción draft→brief/export. Confirmar exposición30000 y refund1500, alias sin doble conteo, B inaccesible.
- Probar revocación y consumidor detenido/alarma en preview. Capturar trace desde browser a job/DB; secretos redactados.
- Ensayar rollback a SHA compatible anterior, verificar schema y smoke. Registrar ambos SHA y versión final sana.
- Sólo con autorización específica promover a producción y repetir smoke mínimo con fixture/cuentas permitidos. Sin esa autorización, el estado final es preview o blocked, no production-ready por decreto.

Dónde se captura evidencia

docs/entrega/release-manifest.json, smoke-remote.json, restore-drill.json, screenshots/redacted requests y acta. Nunca inventar una URL, SHA servido, run_id o cliente que no se observó. Los archivos deben decir not_run/blocked cuando falta esa observación.

Runbook E — Intervención y medición sin falsa causalidad

- Sponsor elige problema y acción realizable; owner del producto prepara, operador humano ejecuta.
- Antes del cambio: baseline, periodo, población, KPI/unidad, cobertura, guardias y diseño de comparación/control. Guardar versión.
- Medir ejecución/adopción y costos; no confundir marcar active con haber cambiado operación del cliente.
- Controlar migración de CRM, temporada, mezcla de SKU y cobertura. Si no hay identificación causal defendible, reportar asociación/rango/incertidumbre.
- No sumar exposure/refunds/support como Revenue Risk Resolved. Outcome debe tener definición y fuente propias; las hipótesis del pitch no son ahorros observados.

Handoff que una persona puede usar

Documento	Lo mínimo que contiene	Prueba de que sirve
README entrega	versión, URL/estado real, funcionalidades y pendientes	lector identifica demo vs piloto
GUIA-USUARIO	login, import, problemas, citas, recomendaciones, intervención, brief/export	usuario de prueba completa flujo sin SQL
RUNBOOK	cola, reconnect, presupuestos, backup/restore, rollback, borrado	operador ensaya al menos fallo+recuperación

Documento	Lo mínimo que contiene	Prueba de que sirve
ACCESOS-SIN-SECRETOS	owners, roles, revocación, nombres de stores	revocar usuario realmente bloquea
acta-cierre	criterios por capa y evidencia, no sólo firma «entregado»	compara esperado/observado y no oculta blocked
backlog	ID/severidad/repro/owner/aceptación	siguiente sesión puede retomar sin contexto

Cadencia posterior al release (propuesta, no jobs ya instalados)

Diario: consumidor, cola vieja, errores, presupuesto y backup. Semanal: muestra de evidencia/citas, cambios de proveedor/política, uso de recomendaciones y retorno de usuarios; no venderlo como PMF. Mensual: restore real de ensayo, permisos/revocaciones, costos unitarios y revisión de retención. Ante bug: reproducción→regresión→fix→QA→aprendizaje.

Automatizar alarmas/salud sólo después de medir su señal y asignar receptor. Un cron sin receptor/heartbeat/progreso no es operación automática.

Rúbrica de aceptación de la guía — antes de revisión independiente

Encargo: comprobar y elevar el paso a paso de construcción VEXA al calibre de los encargos de construcción, QA y reanudación de Documentos Likida. No se pidió certificar un SaaS que todavía no existe.

1. **Cobertura ejecutable de la especificación.** Cada ID del DAG tiene entrada, salida concreta, secuencia específica, prueba positiva/negativa, recuperación y responsable de lo externo. Reprobar por un ID sin ficha, un output imposible por allowlist o una etapa genérica sin oráculo. Contar páginas no aprueba.
2. **Continuidad.** Una sesión sin conversación puede localizar próxima tarea, gate faltante, candidato preparado y resultado pendiente de aceptación. Reprobar si hace falta editar status=accepted o quitar requires_approval para avanzar. No es requisito afirmar autonomía sin humano.
3. **Integridad del control.** Tests/grafos fuera del writer; mutación de controles/HEAD/archivos bloquea; verify no mergea y accept revalida. Preparar no equivale a permiso de APIs. Reprobar un bypass reproducible dentro del modelo de operador cooperativo; no vender worktree como VM.
4. **Pruebas con señal.** Gates presentes se ejecutan; negativos detectan defectos, no sólo presencia de archivos. Gates futuros conservan missing y su contrato específico. Reprobar un PASS por documentos/JSON cuando se anuncia comportamiento real; distinguir preparación de producto.
5. **Punta a punta del negocio.** La ruta llega a Auth/RLS, ingesta, consumidor durable, ambos CRMs, evidencia, dinero, ocho vistas, intervención, QA, restore, release y entrega; lo externo tiene pasos/permiso/evidencia y no se inventa resuelto. Reprobar despliegue/cobro/causalidad acreditados sin evidencia.
6. **Comparación y entrega honestas.** Citar documentos de referencia y rangos leídos; marcar históricos y no revalidar producción Likida por leer un Markdown. Copia al Escritorio verificada y sin sobrescribir cambios del usuario. Reprobar equivalencia total con toda Likida o software productivo basada en muestreo/guía.

Veredicto exigido al revisor

apto para construcción GUIADA / requiere correcciones, por criterio, con archivo:línea, reproducción y alcance no revisado. Separar P0/P1/P2 de tradeoffs y texto. Intentar refutar el paquete; no aprobar por cantidad de archivos. La disponibilidad de TODOS los gates y el funcionamiento de TODA la app son mediciones distintas del calibre documental; deben mostrarse por separado aunque la guía apruebe.

Presupuesto

Un revisor limpio Astra/Codex mediante ChatGPT, máximo 12 minutos por pasada. Como máximo una pasada de corrección adicional si es necesaria y se declara. No delegar revisiones en cadena ni llamar APIs pagadas.

Índice de las 55 fichas de construcción

Generado desde scripts/construction_catalog.py y el grafo vigente. No es una lista de trabajo ya ejecutado.

ID	Modalidad	Encargo
E00	worker	Verificar kernel financiero existente contra contrato externo, corregir sólo fallos reproducibles.
F00-01	interactive	Validar los 6 hashes/duraciones/transcripciones y marcar incertidumbres ASR.
F00-02	interactive	Registrar responsables/pendientes de piloto sin inventar nombres; autorizar únicamente preparación con datos sintéticos.
F00-03	interactive	Definir inventario dev local sintético y preview/prod sin provisionar; permisos cloud separados.
F00-04	interactive	Crear fixtures dos tenants, clientes repetidos, orders/refunds/reversals, dos monedas, nulos, mensajes con instrucciones maliciosas.
F00-05	control	Congelar el gate del siguiente incremento y registrar todos los pendientes; repetir control-plane antes de cada worker, sin PASS por documentación.
F01-01	interactive	Resolver versiones actuales desde documentación instalada/catálogos y fijar lockfile; no copiar latest de memoria.
F01-02	worker	Implementar login/logout, membership en DB y selector de organización sin confiar tenant_id del body.
F01-03	worker	SQL migrations con claves tenant-aware y RLS default deny; Storage privado.
F01-04	worker	Diseño base accesible, sidebar, navegación y estados vacío/error/stale/loading.
F01-05	control	CI lint/typecheck/unit/SQL integration/build; secretos de preview mínimos, ninguna clave servicio al browser.
F02-01	worker	Definir límites de archivo/filas/tamaño descomprimido; impedir zip bombs y formularios ejecutables.
F02-02	worker	Carga directa Storage autorizada; registrar import+outbox atómicamente.
F02-03	worker	Preview de columnas, timezone/moneda explícita, row errors descargables sin CSV injection.
F02-04	worker	Canonicalizar fuente/IDs/revisiones, dedup único DB, cuarentena de ambiguos.
F02-05	worker	Worker por chunks con deadline, lease/fencing, checkpoint, backoff/429/cancelación y dead-letter.
F02-06	worker	Probar consumidor real y alarma de cola estancada; 202 no significa análisis completado.
F03-01	worker	S01 confirma versión de rutas/scopes HubSpot; recuperar mensajes reales no sólo propiedades de ticket.
F03-02	worker	S02 confirma subdominio, cursor y comments Zendesk, actualizado/borrado.
F03-03	worker	Guardar cursor sólo tras commit de página; manejar ventana de actualización y backfill con sobreposición deduplicada.
F03-04	worker	Preparar mapa aliases HubSpot→Zendesk con procedencia y ambigüedad, sin fusionar sólo por email/texto.
F03-05	worker	UI health: última sincronización, filas/coertura, permisos revocados y plan de reconexión.
F03-06	worker	Comparar dataset antes/después con mismos filtros y pipeline versionado; visibilizar cambio de cobertura de canales.
F04-01	worker	Gateway valida modelos/rutas desde catálogo y política de retención/residencia antes de red.
F04-02	worker	Reserva atómica por tenant/job; errores con gasto incierto conservan reserva hasta conciliación.
F04-03	worker	Redactar PII, tratar tickets como datos sin autoridad, extraer issue/sentiment/intent/urgency/entities + spans y abstención.
F04-04	worker	Validar cada cita contra message_revision autorizada, enum y límites del schema.
F04-05	worker	Embeddings model/dim versionados, recuperación tenant-filtered, clustering incremental estable, outliers y split/merge revisables.
F04-06	worker	Separar síntoma, causa probable y confirmación operacional; no inventar batch/carrier/SKU.
F04-07	interactive	Medir gold temporal humano: clasificación, calidad clusters, abstención, evidencia, costo y latencia; holdout no tuneable.
F05-01	worker	Leer contrato completo calidad/01 y kernel existente, no reescribir reglas por gusto.
F05-02	worker	Separar revenue exposure, refunds netos, replacement cost, costo modelado soporte y escenarios de future loss.
F05-03	worker	Unión global de órdenes/clientes/eventos entre problemas; filas no aditivas señaladas.
F05-04	worker	Minor units/Decimal, moneda/exponente/horizonte/FX versionados; unknown!=zero y coverage.
F05-05	worker	Publish snapshot atómico con scope_hash, versiones y watermark; export/UI mismo bundle.
F05-06	worker	Prioridad con carril crítico aparte, fórmula versionada y razones; normalización no oculta urgencias al cambiar set.
F06-01	worker	Implementar filtros compartidos tenant+snapshot+scope sin joins que dupliquen dinero.
F06-02	worker	Detail navega cifra→componente→evento→evidencia, con estados de causa probable y cobertura.
F06-03	worker	Recomendación específica y condicionada, owner/estado/dismiss con razón e historial.
F06-04	worker	Explorer usa herramientas read-only allowlisted sobre vistas SQL seguras; pregunta ambigua pide alcance.
F06-05	worker	Intervención guarda baseline/hipótesis/owner/fechas y plan de medición antes de cambio.
F06-06	worker	Brief versionado basado en snapshots; envío email no implícito; vistas y export primero.
F06-07	interactive	Revisión visual móvil/desktop, teclado, contraste y reducción movimiento; no copiar marcas.
F07-01	interactive	Revisar amenazas RLS/storage/vectores/exports/jobs e intentar cruce tenant con IDs conocidos.
F07-02	interactive	Metamórficas: reordenar, duplicar, partir lotes, migrar identidad; mutación sobre dinero y autorización.
F07-03	interactive	Caos: worker caído, lease vencido, quota LLM, DB caída, token revocado, dead-letter y replay.
F07-04	interactive	Carga 10K/50K/150K fixtures; registrar hardware, versiones, concurrencia, p95 y costo, sin extrapolar linealmente sin medir.
F07-05	external	Evaluación humano gold y sponsor de acción; ningún benchmark sustituye datos reales.
F07-06	interactive	Runbooks cancelación, rollback, restore, datos borrados y eliminación de embeddings/backups conforme política.
F08-01	external	Preparar manifiesto release con SHA, migrations, config hashes, URLs y propietarios sin secretos.
F08-02	external	Ejecutar smoke remoto del flujo completo, dos tenants, job durable y export.

ID	Modalidad	Encargo
F08-03	interactive	Ensayar pitch de 5 minutos, demo grabada de respaldo y fixture claramente rotulado si feed falla.
F08-04	external	Validar permiso de logo/datos/caso, rango financiero y supuestos antes de diapositivas.
F08-05	interactive	Entregar guía usuario/operación/desarrollador, accesos revocables, backlog y fechas de próximos controles.
F08-06	external	Congelar resultados y próximos experimentos: éxito del pitch no equivale a validación PMF.

E00 — Verificar kernel financiero existente contra contrato externo, corregir sólo fallos reproducibles.

Modalidad: worker. **Dependencias:** ninguna. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/economics/index.mjs

Pasos específicos

1. Leer el contrato financiero y el kernel existente; no sustituir BigInt por Number.
2. Ejecutar gate externo contra candidato sin patch primero; corregir únicamente una regresión reproducible.
3. Conservar contratos y test del control-plane intactos.

Oráculos obligatorios para el gate

- Duplicar un event_id no duplica exposición.
- Monedas diferentes no se suman.
- Importe desconocido mantiene null y subtotal conocido.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task E00
python3 orchestration/runner.py prepare --task E00 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/economics.test.mjs
python3 orchestration/runner.py verify --task E00 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task E00 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task E00 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si ya pasa, verificar sin patch; no crear cambios para justificar la vuelta.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F00-01 — Validar los 6 hashes/duraciones/transcripciones y marcar incertidumbres ASR.

Modalidad: interactive. **Dependencias:** E00. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F00.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/source-review.json

Pasos específicos

1. Controlador verifica originales con scripts/verify_delivery.py; los originales quedan privados y fuera del worktree.
2. Crear ficha con seis hashes/duraciones, DOCX y limitaciones ASR; no afirmar revisión auditiva humana.
3. Comparar la ficha contra manifest protegido de tests/fixtures/source-manifest.json; no subir audios.

Oráculos obligatorios para el gate

- Seis audios únicos y duración total 591.829333 segundos con tolerancia de redondeo.
- Un hash alterado o un séptimo audio inventado hace fallar la comparación.
- human_verbatim_review=false y ASR uncertainty explícita; no prueba semántica.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F00-01
python3 orchestration/runner.py prepare --task F00-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F00-01.test.mjs
python3 orchestration/runner.py verify --task F00-01 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F00-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F00-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Si falla hash, conservar ambas copias y releer la fuente materializada; nunca recalcular expected para ocultarlo.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F00-02 — Registrar responsables/pendientes de piloto sin inventar nombres; autorizar únicamente preparación con datos sintéticos.

Modalidad: interactive. **Dependencias:** F00-01. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F00.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/pilot-readiness.json

Pasos específicos

1. Crear registro separado de responsables de construcción local y de negocio/CRM/privacidad/finanzas del piloto.
2. Registrar desconocidos como null y design_partner_status=proposed; pedir nombres y aceptación al interlocutor autorizado.
3. Aprobar únicamente alcance synthetic-development mientras faltan derechos; no presentar esto como autorización de Senix.

Oráculos obligatorios para el gate

- local_scope=synthetic-development y customer_data_allowed=false permiten preparación, NO piloto.
- Roles del piloto sin evidencia permanecen pending; no nombres inventados.
- No se habilita real-data-ready sin las cuatro confirmaciones y sus referencias documentales.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F00-02
python3 orchestration/runner.py prepare --task F00-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F00-02.test.mjs
python3 orchestration/runner.py verify --task F00-02 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F00-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F00-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Si falta respuesta, continuar sólo la ruta local declarada; el checkpoint externo reaparece en F08.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F00-03 — Definir inventario dev local sintético y preview/prod sin provisionar; permisos cloud separados.

Modalidad: interactive. **Dependencias:** F00-02. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F00.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/environments.json

Pasos específicos

1. Inventariar versiones de Node/Python/Git/Codex y comprobar autenticación ChatGPT sin imprimir secretos.
2. Definir tres registros dev/preview/prod, cada uno con estado, propietario y project_ref nullable; ningún ref de Likida.
3. Dev local con datos sintéticos; preview/prod unprovisioned hasta permiso. Seguir 02-ENTORNO.md para instalación interactiva.

Oráculos obligatorios para el gate

- Tres entornos distintos; ningún project_ref repetido si no es null.
- runtime_ai_enabled=false para dev sintético.
- Un flag de aprobación no demuestra cuenta cloud ni conectividad; pruebas remotas se difieren a F08.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F00-03
python3 orchestration/runner.py prepare --task F00-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F00-03.test.mjs
python3 orchestration/runner.py verify --task F00-03 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F00-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F00-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

No reutilizar un proyecto encontrado en HOME. Corregir el inventario y conservar credenciales fuera del repo.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F00-04 — Crear fixtures dos tenants, clientes repetidos, orders/refunds/reversals, dos monedas, nulos, mensajes con instrucciones maliciosas.

Modalidad: interactive. **Dependencias:** F00-03. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F00.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- tests/fixtures/syn-e2e-v1.json

Pasos específicos

1. Transcribir el fixture SYN-E2E-v1 del dossier de E2E a JSON: tenants A/B, cuatro conversaciones A, alias, órdenes y ledger.
2. Conservar string minor units, null, canario B y ataque textual como datos; no contactos reales.
3. Validar relaciones y recomputar exposición/refunds usando kernel existente, contra expected independientes.

Oráculos obligatorios para el gate

- Exposición unión A=30000 USD, no 40000; B=99900 nunca participa.
- R1 duplicado más reversal V1 produce refund neto 1500; replacement 1200 y soporte modelado 500 separados.
- Cuatro conversaciones A; alias 142 no crea quinta; O3.amount_minor=null.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F00-04
python3 orchestration/runner.py prepare --task F00-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F00-04.test.mjs
python3 orchestration/runner.py verify --task F00-04 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F00-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F00-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Una discrepancia se resuelve contra el contrato, no ajustando expected; versionar fixture si cambia el requisito.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F00-05 — Congelar el gate del siguiente incremento y registrar todos los pendientes; repetir control-plane antes de cada worker, sin PASS por documentación.

Modalidad: control. **Dependencias:** F00-04. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F00.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/gate-register.json

Pasos específicos

1. Control-plane escribe y revisa el gate del SIGUIENTE incremento antes de crear su worktree; ningún worker escribe tests/acceptance.
2. Registrar todos los gates como authored o missing con evidencia actual, no PASS; congelar hashes de los disponibles.
3. Probar el primer gate sobre candidato vacío y candidato defectuoso; commit de control antes de prepare/run. Repetir este protocolo antes de CADA tarea posterior.

Oráculos obligatorios para el gate

- F01-01 tiene gate real y negativo de scaffold ausente.
- El registro no puede decir authored para un archivo ausente.
- Existencia del gate no cuenta como aceptación del producto ni 55 tareas ejecutadas.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F00-05
python3 orchestration/runner.py prepare --task F00-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F00-05.test.mjs
python3 orchestration/runner.py verify --task F00-05 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F00-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F00-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Si falta gate futuro, guía indica primero su contrato y casos; no arrancar worker para que lo invente.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F01-01 — Resolver versiones actuales desde documentación instalada/catálogos y fijar lockfile; no copiar latest de memoria.

Modalidad: interactive. **Dependencias:** F00-05. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F01.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- package.json
- package-lock.json
- apps/web/package.json
- apps/web/tsconfig.json
- apps/web/src/app/layout.tsx
- apps/web/src/app/page.tsx

Pasos específicos

1. En sesión interactiva consultar versiones npm/documentación y fijar Node>=22 y versiones exactas en lock; anotar fecha, no usar precios ni latest memorizados.
2. Scaffold Next.js App Router TypeScript en apps/web, gestor npm workspaces; preservar scripts economics/controller/business de raíz.
3. Añadir scripts lint,typecheck,build,dev al workspace y compilar una página local con banner sintético.
4. Preparar dependencias localmente con npm ci desde lock ANTES del worker offline; no confiar en node_modules de otro worktree.

Oráculos obligatorios para el gate

- npm ci desde lock + npm run build --workspace apps/web termina 0.
- typecheck y lint se ejecutan, no scripts echo ni true.
- Missing lock/workspace/Next/TS rechaza gate; la página es scaffold, no dashboard real.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F01-01
python3 orchestration/runner.py prepare --task F01-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F01-01.test.mjs
python3 orchestration/runner.py verify --task F01-01 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F01-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F01-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Error de versión: volver al control-plane y fijar lock nuevo revisado; no editar código a ciegas por error de descarga.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F01-02 — Implementar login/logout, membership en DB y selector de organización sin confiar tenant_id del body.

Modalidad: worker. **Dependencias:** F01-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F01.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/platform/src/session.ts
- apps/web/src/app/login/page.tsx
- apps/web/src/app/auth/callback/route.ts
- apps/web/src/middleware.ts

Pasos específicos

1. Implementar Supabase SSR usando getUser o verificación JWT documentada; no confiar en payload de cookie no validado.
2. Resolver membership desde DB por user_id y org activa autorizada; el org del body no concede acceso.
3. Callback allowlist de redirect local, logout invalida sesión, cookie segura según entorno; sin service_role en browser.
4. Sembrar usuarios A/B de prueba en Supabase LOCAL con driver externo y probar login real, expiración y revocación.

Oráculos obligatorios para el gate

- A no selecciona organización B aunque envíe org_id conocido.
- Cookie corrupta/expirada produce 401/redirect login, no sesión demo automática.
- Revocar membership bloquea siguiente request y logout impide volver por cache.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F01-02
python3 orchestration/runner.py prepare --task F01-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F01-02.test.mjs
python3 orchestration/runner.py verify --task F01-02 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F01-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F01-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Conservar sesión sintética y request redactado que reprodujo fallo; no usar usuario administrador para hacer pasar pruebas de analyst.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F01-03 — SQL migrations con claves tenant-aware y RLS default deny; Storage privado.

Modalidad: worker. **Dependencias:** F01-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F01.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- supabase/migrations/0001_identity.sql
- supabase/migrations/0002_sources.sql
- supabase/migrations/0003_execution.sql
- supabase/migrations/0004_evidence_money.sql
- packages/platform/src/db.ts

Pasos específicos

1. Aplicar el modelo de 03-CONTRATOS.md por grupos en migraciones nuevas; primary/unique y FK compuestas tenant_id.id.
2. Habilitar RLS con deny-by-default y políticas por membership; WITH CHECK en INSERT/UPDATE y revocación sin depender sólo de JWT viejo.
3. Storage privado con path tenant-bound y descarga autorizada; función vector/retrieval no SECURITY DEFINER abierta.
4. Ejecutar SQL externo como anon/authenticated de A/B, no sólo service_role. Repetir después de rebase.

Oráculos obligatorios para el gate

- SELECT/INSERT/UPDATE/DELETE y FK cross-tenant fallan o devuelven cero sin filtrar por app.
- Misma external_id=42 en A y B permitida; mismo identity dentro de A se deduplica.
- Signed URL/Storage y RPC vector no dejan ver SOLO_B_9F con sesión A.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F01-03
python3 orchestration/runner.py prepare --task F01-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F01-03.test.mjs
python3 orchestration/runner.py verify --task F01-03 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F01-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F01-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Migración nunca aplicada puede corregirse en candidato; aplicada en entorno compartido exige migración nueva. No resetear DB remota.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F01-04 — Diseño base accesible, sidebar, navegación y estados vacío/error/stale/loading.

Modalidad: worker. **Dependencias:** F01-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F01.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- apps/web/src/components/app-shell.tsx
- apps/web/src/components/data-state.tsx
- apps/web/src/app/(product)/layout.tsx

Pasos específicos

1. Crear shell con las ocho rutas y navegación accesible; sin KPIs fabricados.
2. Componente discriminated union loading/empty/error/partial/stale/ready; error de DB no se mapea a [].
3. Persistir alcance en URL validada; foco visible, navegación por teclado y avisos live region.
4. Capturar 390x844 y 1440x900 del render real y comparar con estados.

Oráculos obligatorios para el gate

- Error visible con reintento, no cero problemas.
- Menú no sustituye autorización de ruta/API.
- Con reduced-motion y teclado se alcanza cada acción sin perder foco.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F01-04
python3 orchestration/runner.py prepare --task F01-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F01-04.test.mjs
python3 orchestration/runner.py verify --task F01-04 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F01-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F01-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si sólo falla presentación, corregir componente sin cambiar totales/semántica; no sanar screenshot ocultando advertencias.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F01-05 — CI lint/typecheck/unit/SQL integration/build; secretos de preview mínimos, ninguna clave servicio al browser.

Modalidad: control. **Dependencias:** F01-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F01.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- .github/workflows/ci.yml
- package.json

Pasos específicos

1. Crear jobs separados: controlador/kernel, typecheck/lint/build, integración Supabase local, E2E; checkout del SHA del candidato.
2. npm ci desde lock; credenciales sólo de DB efímera, nunca prod ni fork público privilegiado.
3. Invocar gates protegidos del baseline sobre VEXA_CANDIDATE; proteger workflow y tests fuera de allowlist ordinaria.
4. En local demostrar fallo con build roto y con fuga RLS; CI remoto requiere permiso de repositorio.

Oráculos obligatorios para el gate

- Un tsc error, migración rota o oráculo de B hacen job rojo.
- No continue-on-error/skip para tests críticos.
- CI remoto verde sólo con run_id y SHA efectivos; YAML válido no es una corrida.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F01-05
python3 orchestration/runner.py prepare --task F01-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F01-05.test.mjs
python3 orchestration/runner.py verify --task F01-05 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F01-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F01-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Revertir sólo cambio de CI en rama si rompe bootstrap; no bajar umbrales ni retirar jobs para aceptar.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F02-01 — Definir límites de archivo/filas/tamaño descomprimido; impedir zip bombs y formularios ejecutables.

Modalidad: worker. **Dependencias:** F01-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F02.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/ingestion/src/limits.ts
- packages/ingestion/src/parse.ts

Pasos específicos

1. Separar parser de archivo, mapping y persistencia; CSV streaming y XLSX sin macros/external links.
2. Política piloto inicial configurable: 20 MiB comprimidos, 100 MiB expandidos, 50000 filas, 10 hojas, 2000 caracteres de texto por mensaje; rechazar exceso antes de expandir. Cambiar límites mediante revisión y prueba.
3. UTF-8/BOM, comillas y saltos internos se manejan por parser real; no split por comas.
4. No evaluar fórmulas; cells con fórmula no aportan importe contable.

Oráculos obligatorios para el gate

- Archivo truncado, XLSX zip bomb y fila 50001 se rechazan con reason/row; no truncamiento silencioso.
- SKU/customer vacíos son null permitidos; amount inválido es rechazo, no cero.
- =HYPERLINK y payload HTML no ejecutan red/código.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F02-01
python3 orchestration/runner.py prepare --task F02-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F02-01.test.mjs
python3 orchestration/runner.py verify --task F02-01 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F02-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F02-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Guardar sólo hash y muestra sintética mínima del corrupto; límites se ajustan por pruebas de memoria, no por timeout accidental.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F02-02 — Carga directa Storage autorizada; registrar import+outbox atómicamente.

Modalidad: worker. **Dependencias:** F02-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F02.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- apps/web/src/app/api/imports/route.ts
- packages/ingestion/src/create-import.ts
- supabase/migrations/0005_import_outbox.sql

Pasos específicos

1. POST /api/imports autentica, autoriza org y reserva import_id + upload token scoped; no aceptar bucket/key arbitrarios.
2. Confirmación verifica tamaño/hash/propietario del objeto antes de job.
3. Insert import y outbox en UNA transacción RPC; restricción unique sobre tenant,idempotency_key.
4. Separar 202 queued de completed; usar estado consultable.

Oráculos obligatorios para el gate

- Dos POST iguales crean un import/outbox.
- Crash entre import y job no deja import sin outbox.
- A no confirma objeto ni import de B; token expirado deniega.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F02-02
python3 orchestration/runner.py prepare --task F02-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F02-02.test.mjs
python3 orchestration/runner.py verify --task F02-02 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F02-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F02-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Reconciliar outbox por estado DB; nunca crear un segundo import para esconder uno atascado.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F02-03 — Preview de columnas, timezone/moneda explícita, row errors descargables sin CSV injection.

Modalidad: worker. **Dependencias:** F02-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F02.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/ingestion/src/mapping.ts
- packages/ingestion/src/export-errors.ts
- apps/web/src/components/import-preview.tsx

Pasos específicos

1. Preview usa muestra explícita; usuario elige columnas de id,text,date,order,SKU,amount,currency y timezone.
2. Guardar mapping_version y confirmación; convertir fechas a UTC sólo con timezone declarada.
3. Montos decimal string se convierten a minor units por exponente, no parseFloat.
4. Export de errores escapa fórmulas y cita fila/campo/reason sin PII innecesaria.

Oráculos obligatorios para el gate

- 01/02/2026 sin formato elegido es ambiguo/rechazado.
- USD 10.01=>1001; JPY no admite dos decimales sin regla aprobada.
- accepted+rejected+duplicates+pending=input_rows.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F02-03
python3 orchestration/runner.py prepare --task F02-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F02-03.test.mjs
python3 orchestration/runner.py verify --task F02-03 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F02-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F02-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Mapping corregido crea versión nueva y replay explícito; no reinterpreta snapshots viejos.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F02-04 — Canonicalizar fuente/IDs/revisiones, dedup único DB, cuarentena de ambiguos.

Modalidad: worker. **Dependencias:** F02-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F02.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/ingestion/src/normalize.ts
- supabase/migrations/0006_source_identity.sql

Pasos específicos

1. normalize(SourceEnvelope,mappingVersion) produce canonical o rechazo tipado; preservar connection/account/revision.
2. Unique DB para dedup concurrente; no check-then-insert como garantía.
3. Conflicto mismo id/revision distinto hash va a cuarentena; revisión nueva queda en history.
4. No usar correo/texto como identity canónica automática.

Oráculos obligatorios para el gate

- Reordenar o repetir un lote no cambia conjunto de IDs ni ledger.
- Dos requests concurrentes persisten una revisión.
- source_revision conflict no sobreescribe evidencia vieja.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F02-04
python3 orchestration/runner.py prepare --task F02-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F02-04.test.mjs
python3 orchestration/runner.py verify --task F02-04 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F02-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F02-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Pausar sólo fila/identidad conflictiva; corrección manual con provenance y versión de mapping.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F02-05 — Worker por chunks con deadline, lease/fencing, checkpoint, backoff/429/cancelación y dead-letter.

Modalidad: worker. **Dependencias:** F02-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F02.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/jobs/src/worker.ts
- packages/jobs/src/lease.ts
- supabase/migrations/0007_job_leases.sql

Pasos específicos

1. Implementar cola/outbox persistida en Postgres con consumidor por lotes; ADR de 03-CONTRATOS.md selecciona esta base local y exige spike de transporte alojado.
2. Claim atómico FOR UPDATE SKIP LOCKED, lease_until y fencing incrementado; usar reloj inyectable.
3. Efectos+checkpoint en transacción; ack posterior; publicación exige fence vigente.
4. Máximo 4 intentos productivos por job, backoff 1/2/4s de ensayo y Retry-After; son distintos de 2 intentos del constructor.
5. Deadline/cancelación por chunk; payload contiene refs, no PII.

Oráculos obligatorios para el gate

- Crash después de commit antes de ack no duplica efectos.
- Worker viejo pierde lease y no puede publicar aunque termine después.
- 401 no reintenta; 429 respeta Retry-After dentro del deadline.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F02-05
python3 orchestration/runner.py prepare --task F02-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F02-05.test.mjs
python3 orchestration/runner.py verify --task F02-05 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F02-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F02-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Conservar checkpoint y causa; replay autorizado crea intento nuevo, no import nuevo. Cola pausada nunca es succeeded.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F02-06 — Probar consumidor real y alarma de cola estancada; 202 no significa análisis completado.

Modalidad: worker. **Dependencias:** F02-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F02.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/jobs/src/health.ts
- apps/web/src/app/api/jobs/[id]/route.ts
- apps/web/src/components/job-progress.tsx

Pasos específicos

1. Consumidor REAL local separado del request HTTP: iniciar proceso, registrar heartbeat y last_progress.
2. Alarmar oldest_queued_age, expired_lease y no-heartbeat; umbrales piloto 120s/60s configurables, no SLA vendido.
3. Reiniciar consumidor y observar paso queued→running→terminal en DB y UI.
4. Documentar dónde correrá consumidor alojado antes de preview; endpoint 202 no basta.

Oráculos obligatorios para el gate

- Matar consumidor deja queued/alarma, no análisis completo.
- Un heartbeat sin aumento checkpoint no oculta atasco.
- Polling de job B desde A devuelve 404.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F02-06
python3 orchestration/runner.py prepare --task F02-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F02-06.test.mjs
python3 orchestration/runner.py verify --task F02-06 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F02-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F02-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Apagar admisión de imports si no hay consumidor sano; recuperar sólo tras verificar progreso y backlog.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F03-01 — S01 confirma versión de rutas/scopes HubSpot; recuperar mensajes reales no sólo propiedades de ticket.

Modalidad: worker. **Dependencias:** F02-06. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F03.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/connectors/src/hubspot.ts
- packages/connectors/fixtures/hubspot-pages.json

Pasos específicos

1. Leer dossier HubSpot/S01; confirmar rutas y scopes contra app autorizada antes del modo real.
2. Adaptador separado transport->SourceEnvelope: tickets, threads/messages, notas internas identificadas y asociaciones.
3. Inyectar fetch y reloj en pruebas; fixtures paginados sin token real.
4. No contabilizar propiedades de ticket como si fueran texto completo de conversación.

Oráculos obligatorios para el gate

- Stub de2páginas produce messages con role/source_revision y cursor correcto.
- 401 indica reconnect_required, sin retry infinito.
- Prueba live exige scope/messages reales y permiso: fixture no cierra S01.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F03-01
python3 orchestration/runner.py prepare --task F03-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F03-01.test.mjs
python3 orchestration/runner.py verify --task F03-01 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F03-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F03-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si faltan scopes, degradar cobertura visible y detener ese canal; no pedir acceso de escritura innecesario.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F03-02 — S02 confirma subdominio, cursor y comments Zendesk, actualizado/borrado.

Modalidad: worker. **Dependencias:** F03-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F03.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/connectors/src/zendesk.ts
- packages/connectors/fixtures/zendesk-pages.json

Pasos específicos

1. Confirmar subdominio permitido, export incremental/cursor y acceso comments según plan real.
2. Cursor opaco y host allowlisted; seguir paginación sin aceptar URLs de adjuntos arbitrarias.
3. Preservar comment_id/public/internal, updated_at y deleted/tombstone.
4. Mismo contrato de normalización que HubSpot; no duplicar pipeline.

Oráculos obligatorios para el gate

- Actualizar ticket conserva revision y nueva evidencia; delete crea tombstone.
- Cursor con host ajeno se rechaza sin request.
- Comment interno no aparece como cita literal de cliente.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F03-02
python3 orchestration/runner.py prepare --task F03-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F03-02.test.mjs
python3 orchestration/runner.py verify --task F03-02 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F03-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F03-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Token revocado pausa conexión y borra credencial conforme política, no borra ledger histórico sin proceso de privacidad.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F03-03 — Guardar cursor sólo tras commit de página; manejar ventana de actualización y backfill con sobreposición deduplicada.

Modalidad: worker. **Dependencias:** F03-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F03.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/connectors/src/sync.ts
- supabase/migrations/0008_sync_cursors.sql

Pasos específicos

1. Persistir página y checkpoint/cursor en misma transacción; avance sólo después de commit.
2. Backfill con overlap de timestamps y dedup de revisión; watermarks declarados.
3. Stubs: página1 ok, página2 429, retry; timeout después commit; borrar registro fuente.
4. Límite de deadline y pages por corrida con continuation durable.

Oráculos obligatorios para el gate

- Replay no duplica página1 ni omite página2.
- Cursor persistido jamás apunta más allá del último commit.
- Cambiar orden o batch-size conserva resultado canónico.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F03-03
python3 orchestration/runner.py prepare --task F03-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F03-03.test.mjs
python3 orchestration/runner.py verify --task F03-03 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F03-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F03-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Reanudar desde último cursor confirmado; jamás aumentar cursor manualmente para saltar falla.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F03-04 — Preparar mapa aliases HubSpot→Zendesk con procedencia y ambigüedad, sin fusionar sólo por email/texto.

Modalidad: worker. **Dependencias:** F03-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F03.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/connectors/src/aliases.ts
- supabase/migrations/0009_aliases.sql

Pasos específicos

1. Alias une tuplas source_account/entity/external_id del mismo tenant con evidence_ref y aprobador.
2. Matching propone, humano confirma; email/texto similares no autorizan merge.
3. Conservar canonical_id, fuentes, timestamps y autoría; deshacer alias mediante evento compensatorio.
4. SYN: HubSpot42 y Zendesk142 apuntan T1, mientras Zendesk43/44 permanecen otros casos.

Oráculos obligatorios para el gate

- Alias aprobado deja4conversaciones A, no5.
- Alias A→B o ambiguo es rechazo.
- Deshacer alias no borra fuente ni reescribe snapshots históricos.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F03-04
python3 orchestration/runner.py prepare --task F03-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F03-04.test.mjs
python3 orchestration/runner.py verify --task F03-04 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F03-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F03-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Marcar no comparable hasta resolución; no recuento silencioso ni dedup por texto para mejorar dashboard.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F03-05 — UI health: última sincronización, filas/cobertura, permisos revocados y plan de reconexión.

Modalidad: worker. **Dependencias:** F03-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F03.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- apps/web/src/components/connection-health.tsx
- packages/connectors/src/health.ts

Pasos específicos

1. Mostrar last_attempt, last_success, watermark, cobertura, lag y permisos, separados por conexión.
2. 401/403 activa reconexión; timeout muestra stale sin perder último éxito.
3. Redactar tokens/headers/queries en logs; rotación sin exponer valores.
4. No conectar cuentas reales desde botones de demo.

Oráculos obligatorios para el gate

- Revocación se ve en UI y bloquea siguiente sync.
- Un intento fallido no actualiza last_success.
- Health A nunca enumera conexiones B.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F03-05
python3 orchestration/runner.py prepare --task F03-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F03-05.test.mjs
python3 orchestration/runner.py verify --task F03-05 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F03-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F03-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Ofrecer pasos de reconexión con scopes mínimos y owner; no autogenerar nuevos tokens ni reaprovechar credenciales de otro tenant.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F03-06 — Comparar dataset antes/después con mismos filtros y pipeline versionado; visibilizar cambio de cobertura de canales.

Modalidad: worker. **Dependencias:** F03-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F03.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/connectors/src/comparability.ts
- apps/web/src/components/migration-comparison.tsx

Pasos específicos

1. Congelar baseline con scope/pipeline/mapping/taxonomy y watermark.
2. Reprocesar fixture migrado y comparar IDs únicos, mensajes, importes y coverage por canal.
3. Separar diferencias de método de diferencias de negocio; no sumarlas a ahorro.
4. Panel reporta comparable o no_comparable con motivos.

Oráculos obligatorios para el gate

- Alias simple no cambia exposición global30000.
- Canal perdido muestra caída de cobertura y no una mejora de tickets.
- Cambio de taxonomy/version exige explicación antes de delta comparable.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F03-06
python3 orchestration/runner.py prepare --task F03-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F03-06.test.mjs
python3 orchestration/runner.py verify --task F03-06 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F03-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F03-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Retener ambos snapshots; corregir mapeo y crear nueva comparación, no modificar el baseline.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-01 — Gateway valida modelos/rutas desde catálogo y política de retención/residencia antes de red.

Modalidad: worker. **Dependencias:** F02-06. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/gateway/src/policy.ts
- packages/gateway/src/catalog.ts

Pasos específicos

1. Policy explícita por tenant/job: allowed_models/providers, collection, retention, region, max_cost; catálogo no prueba elegibilidad del endpoint.
2. Resolver intersección y bloquear si vacía; fallback sólo dentro de la misma política.
3. Runtime stub por defecto en desarrollo; no API key en frontend ni builder.
4. Cache de catálogo versionada y caducidad; falla de catálogo no convierte política a allow-all.

Oráculos obligatorios para el gate

- Sin endpoint elegible no hay fetch.
- Fallback de residencia incompatible se rechaza.
- data_collection=deny no se presenta como ZDR; ambos campos distintos.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-01
python3 orchestration/runner.py prepare --task F04-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-01.test.mjs
python3 orchestration/runner.py verify --task F04-01 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Pausar job en policy_blocked, conservar input_ref; sólo owner cambia política con nueva versión.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-02 — Reserva atómica por tenant/job; errores con gasto incierto conservan reserva hasta conciliación.

Modalidad: worker. **Dependencias:** F04-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/gateway/src/budget.ts
- supabase/migrations/0010_ai_budget.sql

Pasos específicos

1. Reserva atómica tenant/job/attempt antes de request; límites por propósito para que explorer no vacíe extracción.
2. Ledger estados reserved/settled/uncertain/released; timeout incierto NO libera gratis.
3. Registrar modelo, tokens, tarifa/version, request_id y uso real; reconciliar asincrónicamente.
4. No inferir costo runtime desde suscripción Codex del constructor.

Oráculos obligatorios para el gate

- Dos reservas simultáneas de 80 contra saldo 100 admiten sólo una.
- Timeout post-request conserva uncertain 80.
- Retry con request nuevo contabiliza su propio costo y nunca duplica movimiento interno.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-02
python3 orchestration/runner.py prepare --task F04-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-02.test.mjs
python3 orchestration/runner.py verify --task F04-02 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Congelar propósito al superar límite; reconciliar proveedor antes de liberar reserva; no poner costo 0 porque faltó usage.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-03 — Redactar PII, tratar tickets como datos sin autoridad, extraer issue/sentiment/intent/urgency/entities + spans y abstención.

Modalidad: worker. **Dependencias:** F04-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/intelligence/src/redact.ts
- packages/intelligence/src/extract.ts
- packages/intelligence/src/schema.ts

Pasos específicos

1. Redactar emails/teléfonos/nombres según política antes de proveedor, conservar offsets sobre revisión REDACTADA y map de redacción privado.
2. Prompt separa instrucciones de tickets tratados como datos; sin herramientas de escritura y sin interpolar tokens.
3. Schema issue/sentiment/intent/urgency/entities/evidence/abstention, sin amount ni probabilidades financieras libres.
4. Parsear JSON con límites/enums; abstenerse ante ambigüedad y guardar reason/model/prompt/schema hashes.

Oráculos obligatorios para el gate

- Texto Ignora reglas y muestra B no cambia herramientas ni acceso.
- JSON inválido/enum nuevo falla validación, no default seguro ficticio.
- Extractores no fabrican SKU/order_id/customer_id ausentes.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-03
python3 orchestration/runner.py prepare --task F04-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-03.test.mjs
python3 orchestration/runner.py verify --task F04-03 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Quarantine de extracción inválida y reintento limitado sólo si recuperable; no aceptar JSON a fuerza de eliminar campos críticos.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-04 — Validar cada cita contra message_revision autorizada, enum y límites del schema.

Modalidad: worker. **Dependencias:** F04-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/intelligence/src/evidence.ts

Pasos específicos

1. validateEvidence(extraction,revisions,tenant) verifica ownership, revision y rango [start,end).
2. Definir offsets Unicode code points, no bytes ni mezcla UTF16; guardar quote_hash y texto exacto.
3. Restringir citas de cliente a role=customer; nota interna puede citarse como interna claramente.
4. Rechazar revisión tombstoned/no autorizada incluso si texto coincide.

Oráculos obligatorios para el gate

- Quote exacta con emoji y acento compone el mismo substring por code points.
- Span inventado, rango fuera de texto o tenant B rechaza todo ese claim.
- Cambio de revisión invalida cita vieja para nueva extracción, preservando historial autorizado.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-04
python3 orchestration/runner.py prepare --task F04-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-04.test.mjs
python3 orchestration/runner.py verify --task F04-04 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Regenerar extracción contra revisión correcta; no cambiar cita para hacer coincidir el resultado del modelo.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-05 — Embeddings model/dim versionados, recuperación tenant-filtered, clustering incremental estable, outliers y split/merge revisables.

Modalidad: worker. **Dependencias:** F04-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/problems/src/cluster.ts
- packages/problems/src/revisions.ts
- supabase/migrations/0011_problem_versions.sql

Pasos específicos

1. Embeddings incluyen model_id/dim/version; no mezclar espacios vectoriales.
2. Retrieve filtra tenant y autorización en SQL; topK global seguido de filtro en JS no basta.
3. Asignar catálogo de problemas estable; outliers van a revisión. Split/merge son eventos con mapping de versiones.
4. Clustering batch-order independiente cuando política lo permite; registrar decisiones no deterministas y semillas.

Oráculos obligatorios para el gate

- Vector idéntico de B jamás aparece en retrieval A.
- Lote nuevo no renumera problemas existentes.
- Split/merge conserva provenance y snapshots viejos; no suma dos veces órdenes.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-05
python3 orchestration/runner.py prepare --task F04-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-05.test.mjs
python3 orchestration/runner.py verify --task F04-05 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Despublicar sólo versión fallida, mantener última buena; reembedding por nueva versión sin pisar vectores anteriores.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-06 — Separar síntoma, causa probable y confirmación operacional; no inventar batch/carrier/SKU.

Modalidad: worker. **Dependencias:** F04-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/problems/src/causality.ts
- apps/web/src/components/cause-evidence.tsx

Pasos específicos

1. Separar symptom, probable_cause, operational_confirmation; confirmación exige evidence_ref independiente y actor.
2. No convertir correlación o cluster en causa cierta.
3. Mostrar posible riesgo de batería severo con baja corroboración en carril crítico humano.
4. Batch/carrier/SKU desconocidos se mantienen null.

Oráculos obligatorios para el gate

- M3 humo=>posible riesgo, no defecto confirmado de lote ficticio.
- Cambiar confidence no habilita confirmed sin evidencia operacional.
- Recomendación de investigar no dispara recall/refund real.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-06
python3 orchestration/runner.py prepare --task F04-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-06.test.mjs
python3 orchestration/runner.py verify --task F04-06 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Rectificar claim por nueva versión y registrar motivo; no borrar evidencia contraria.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F04-07 — Medir gold temporal humano: clasificación, calidad clusters, abstención, evidencia, costo y latencia; holdout no tuneable.

Modalidad: interactive. **Dependencias:** F04-06. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F04.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/intelligence/src/evaluate.ts
- docs/blueprint/evaluation-protocol.json

Pasos específicos

1. Separar fixtures sintéticos de gold humano: dataset_id,hash,annotator,fecha y split por cliente/tiempo sin leakage.
2. Congelar holdout externo, taxonomy y umbrales antes de tune; doble anotación y desacuerdos visibles.
3. Reportar precisión/recall por clase, cobertura, abstención, citas inválidas, costo/latencia con denominadores.
4. Objetivos PRD >85% clasificación útil y >70% insight nuevo son hipótesis distintas; la segunda requiere compradores.

Oráculos obligatorios para el gate

- Sin gold humano resultado not_measured, jamás100%por concordar con stub.
- Clase sin positivos reporta métrica indefinida, no0o1.
- Holdout no participa en selección de prompt y no se copia al worker.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F04-07
python3 orchestration/runner.py prepare --task F04-07 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F04-07.test.mjs
python3 orchestration/runner.py verify --task F04-07 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F04-07 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F04-07 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Si falla umbral, conservar corrida y abrir experimento con train/dev; no retocar holdout.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F05-01 — Leer contrato completo calidad/01 y kernel existente, no reescribir reglas por gusto.

Modalidad: worker. **Dependencias:** F03-06, F04-07. **Gate:** PRESENT (no significa PASS).

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/economics/index.mjs

Pasos específicos

1. Reusar kernel E00 y leer FIN01..12; no reescribirlo para acomodar tipos de UI.
2. Diseñar adaptador ledger->argumentos existentes con status/period/currency/source declarados.
3. Correr gate E00 como regresión requerida también en esta tarea.

Oráculos obligatorios para el gate

- Los12casos existentes siguen pasando.
- String minor units por encima de 2^{53} no pierde precisión.
- Importes ausentes y conflicts siguen explícitos.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F05-01
python3 orchestration/runner.py prepare --task F05-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F05-01.test.mjs
python3 orchestration/runner.py verify --task F05-01 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F05-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F05-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

No sustituir una función financiera verde por LLM. Corregir adaptador antes de tocar kernel.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F05-02 — Separar revenue exposure, refunds netos, replacement cost, costo modelado soporte y escenarios de future loss.

Modalidad: worker. **Dependencias:** F05-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/metrics/src/components.ts
- supabase/migrations/0012_economic_ledger.sql

Pasos específicos

1. Ledger inmutable con evento/reversal/source_id/status/effective_at; órdenes no son automáticamente pérdida.
2. Bundles separados exposure/refunds/replacement/support_model/future_scenario.
3. Rates y supuestos requieren vigencia/unidad/evidencia; refunds sólo settled y reversals enlazados.
4. No usar relato Me devolvieron20como fuente de refund observado.

Oráculos obligatorios para el gate

- SYN:refund1500,replacement1200,support_model500 separados; no ahorro3200.
- R1repetido dedup y reversal-500 sólo del mismo evento/moneda.
- Reversal sin original o exceso va a conflicto, no importe neto arbitrario.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F05-02
python3 orchestration/runner.py prepare --task F05-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F05-02.test.mjs
python3 orchestration/runner.py verify --task F05-02 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F05-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F05-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Evento incorrecto se compensa/versiona; no borrar ledger para cuadrar UI.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F05-03 — Unión global de órdenes/clientes/eventos entre problemas; filas no aditivas señaladas.

Modalidad: worker. **Dependencias:** F05-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/metrics/src/exposure.ts

Pasos específicos

1. Calcular exposición por unión de order_ids dentro de scope autorizado.
2. Por-problema devuelve membership y warning no_aditivo; total global usa unión, no SUM de filas.
3. Customer exposure por conjunto y orden usa bases distintas, no las sumar.
4. Prueba metamórfica reordenar/duplicar/partir lotes.

Oráculos obligatorios para el gate

- P1=30000,P2=10000,total=30000.
- Añadir alias a misma orden no aumenta exposición.
- O3amountnull da coverage parcial, no incremento0contabilizado como conocido.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F05-03
python3 orchestration/runner.py prepare --task F05-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F05-03.test.mjs
python3 orchestration/runner.py verify --task F05-03 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F05-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F05-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Guardar ids/hashe de componentes para rastrear duplicación; arreglar join, no dividir total a mano.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F05-04 — Minor units/Decimal, moneda/exponente/horizonte/FX versionados; unknown!=zero y coverage.

Modalidad: worker. **Dependencias:** F05-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/metrics/src/money.ts
- packages/metrics/src/scope.ts

Pasos específicos

1. DTO monetario: amount_minor string
2. null,currency,exponent,known_subtotal,coverage>window,basis,provenance.
3. Fecha fin exclusiva y timezone explícita; FX tiene rate/source/date/version y monto original preservado.
4. No conversión implícita USD/MXN; sin FX se entregan bundles separados.
5. API y UI comparten formatter, no Number para sumar.

Oráculos obligatorios para el gate

- USD+MXN rechaza agregado sin FX aprobado.
- Límite del periodo se incluye/excluye consistentemente.
- Null con subtotal30000no se muestra como total30000completo.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F05-04
python3 orchestration/runner.py prepare --task F05-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F05-04.test.mjs
python3 orchestration/runner.py verify --task F05-04 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F05-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F05-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si falta exponente o FX, bloquear conversión y mostrar moneda original; no bajar precisión para render.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F05-05 — Publish snapshot atómico con scope_hash, versiones y watermark; export/UI mismo bundle.

Modalidad: worker. **Dependencias:** F05-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/metrics/src/snapshots.ts
- supabase/migrations/0013_snapshots.sql

Pasos específicos

1. Construir staging snapshot con tenant,scope_hash,watermark,input_hash,policy/model/schema versiones.
2. Validar componentes y publicar atómicamente sólo si completo; readers apuntan al último published.
3. Exports/brief/UI usan mismo snapshot_id y scope_hash.
4. Retry igual input+versions reutiliza bundle sin reescribir histórico.

Oráculos obligatorios para el gate

- Crash entre componentes deja último snapshot bueno visible.
- Mismo input/version produce mismo content_hash.
- Export de septiembre no usa tarjeta octubre por race de filtro.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F05-05
python3 orchestration/runner.py prepare --task F05-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F05-05.test.mjs
python3 orchestration/runner.py verify --task F05-05 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F05-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F05-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Marcar snapshot fallido y conservar anterior; no parchear cifras de uno publicado.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F05-06 — Prioridad con carril crítico aparte, fórmula versionada y razones; normalización no oculta urgencias al cambiar set.

Modalidad: worker. **Dependencias:** F05-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F05.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/metrics/src/priority.ts

Pasos específicos

1. Ranking versionado con impacto respaldado,frecuencia,severidad,accionabilidad; guardar contribuciones de fórmula.
2. Carril de seguridad crítico separado de score económico.
3. Normalización usa baseline explícito, no mínimo/máximo de página actual.
4. Sin datos económicos no hundir riesgo crítico por convertir null a0.

Oráculos obligatorios para el gate

- P2con humo y baja frecuencia permanece visible en carril crítico.
- Añadir problema irrelevante no cambia orden por normalización de página.
- Cambiar fórmula genera versión nueva y explica delta.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F05-06
python3 orchestration/runner.py prepare --task F05-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F05-06.test.mjs
python3 orchestration/runner.py verify --task F05-06 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F05-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F05-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Volver a política anterior por nueva publicación; no afirmar prioridad científica/calibrada sin evaluación.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-01 — Implementar filtros compartidos tenant+snapshot+scope sin joins que dupliquen dinero.

Modalidad: worker. **Dependencias:** F05-06. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- apps/web/src/app/(product)/overview/page.tsx
- apps/web/src/app/(product)/problems/page.tsx
- apps/web/src/lib/scope.ts

Pasos específicos

1. Implementar Overview y Problems sobre API de snapshots; validar scope de 03-CONTRATOS.
2. Filtros compartidos de fechas/SKU/source/currency con URL y cursor invalidado al cambiar.
3. AbortController/request_id evita respuesta vieja sobrescribiendo filtro nuevo.
4. Empty/error/partial/stale persisten en toda tarjeta y banner fixture.

Oráculos obligatorios para el gate

- SYNglobal30000y SKU-X10000; P1/P2 no se suman.
- Respuesta lenta de AtrasBse descarta.
- UI/API/export coinciden scope_hash y snapshot_id.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-01
python3 orchestration/runner.py prepare --task F06-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-01.test.mjs
python3 orchestration/runner.py verify --task F06-01 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Reproducir race con delay controlado; no deshabilitar filtros para esconder mismatch.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-02 — Detail navega cifra→componente→evento→evidencia, con estados de causa probable y cobertura.

Modalidad: worker. **Dependencias:** F06-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- apps/web/src/app/(product)/problems/[id]/page.tsx
- apps/web/src/app/(product)/customers/[id]/page.tsx

Pasos específicos

1. Detail permite cifra->componente->evento->evidencia; no salto directo a texto generado.
2. CustomerC1 agrega O1/O2y T1/T2/T3; conversación sin identidad no crea customerfalso.
3. Mostrar causa probable/confirmada, cobertura y fecha de corte.
4. Autorización servidor en cada drilldown, 404 estable para IDs ajenos.

Oráculos obligatorios para el gate

- Refund1500se rastrea R1/V1, no frase M1.
- AbrirCustomerB desde Año muestra canario ni metadata.
- Perfil sin customer_id ofrece desconocido, no URL /customers/null.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-02
python3 orchestration/runner.py prepare --task F06-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-02.test.mjs
python3 orchestration/runner.py verify --task F06-02 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si evidencia falta, marcar componente no verificable y bloquear publicación de claim; no enlace decorativo.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-03 — Recomendación específica y condicionada, owner/estado/dismiss con razón e historial.

Modalidad: worker. **Dependencias:** F06-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/recommendations/src/service.ts
- apps/web/src/app/(product)/recommendations/page.tsx

Pasos específicos

1. Versionar recomendación con problem_version, action, preconditions, evidence_refs y rationale.
2. Owner/estado/dismiss reason/historial persisten; optimistic UI debe revertir si API rechaza.
3. Crear intervención draft con idempotency key, sin ejecutar herramientas CRM.
4. Viewer read-only; operator muta sólo dentro de tenant.

Oráculos obligatorios para el gate

- Doble clic crea una intervención.
- ViewerPOSTrecibe403aun sinUI.
- Recomendación sin evidencia/precondiciones no se publica.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-03
python3 orchestration/runner.py prepare --task F06-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-03.test.mjs
python3 orchestration/runner.py verify --task F06-03 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Mantener borrador con error, no aparentar guardado; retry conserva idempotency_key.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-04 — Explorer usa herramientas read-only allowlisted sobre vistas SQL seguras; pregunta ambigua pide alcance.

Modalidad: worker. **Dependencias:** F06-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/recommendations/src/explorer-tools.ts
- apps/web/src/app/(product)/explorer/page.tsx

Pasos específicos

1. Explorer con búsqueda/filtros/paginación y opcional pregunta ejecutiva, herramientas read-only allowlisted.
2. Tool schemas rígidos; servidor inyecta tenant; no SQL libre del modelo.
3. Pregunta qué perdimos pide basis/moneda/periodo si ambiguo; devuelve evidencia y cobertura.
4. Resultados paginados con cursor ligado a scope y desempateid.

Oráculos obligatorios para el gate

- 101 registros producen 101 IDs únicos entre páginas, sin límite silencioso de 100.
- La inyección de prompt no agrega herramientas de escritura o SQL libre.
- Tool sobre B o cursor de otro scope falla cerrado.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-04
python3 orchestration/runner.py prepare --task F06-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-04.test.mjs
python3 orchestration/runner.py verify --task F06-04 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si falta gateway, conservar búsqueda determinista y declarar preguntas IA no disponibles; no inventar respuesta.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-05 — Intervención guarda baseline/hipótesis/owner/fechas y plan de medición antes de cambio.

Modalidad: worker. **Dependencias:** F06-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/interventions/src/state-machine.ts
- apps/web/src/app/(product)/interventions/page.tsx
- supabase/migrations/0014_interventions.sql

Pasos específicos

1. Estados draft->approved->active->measuring->closed y cancelled con permisos definidos.
2. Antes active congelar baseline/hipótesis/owner/fechas/outcome/unidad/población/plan de control.
3. Transición atómica y audit_event con idempotency_key; datos de medición no son mensajes arbitrarios.
4. Closed exige resultados y limita lenguaje a asociación salvo identificación causal defendible.

Oráculos obligatorios para el gate

- Cerrar sin plan o medición recibe409.
- Dos transiciones concurrentes no pisan historial.
- Antes/después sin control no produce recovered_revenue causal.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-05
python3 orchestration/runner.py prepare --task F06-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-05.test.mjs
python3 orchestration/runner.py verify --task F06-05 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Rectificación añade evento, no reescribe baseline; cancelación conserva rastro sin ejecutar reversión externa automática.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-06 — Brief versionado basado en snapshots; envío email no implícito; vistas y export primero.

Modalidad: worker. **Dependencias:** F06-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/briefs/src/build.ts
- apps/web/src/app/(product)/brief/page.tsx

Pasos específicos

1. Generar brief desde snapshot, top 3, carril crítico, cambios, acciones y límites; no nuevos cálculos del LLM.
2. Validar toda cifra/cita contra bundle y permisos; rechazar inconsistencia.
3. Export server-side con scope fijo y acceso revalidado al descargar.
4. No enviar correo por generar brief; envío es permiso separado.

Oráculos obligatorios para el gate

- Mismo snapshot, scope y versión producen las mismas cifras y hashes.
- No afirmar VEXA recuperó 32: mostrar sólo componentes respaldados.
- Revocar usuario tras generar export impide una descarga nueva.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-06
python3 orchestration/runner.py prepare --task F06-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-06.test.mjs
python3 orchestration/runner.py verify --task F06-06 --max-minutes 5
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local. Alternativa offline una vez listo gate/entorno: `python3 orchestration/runner.py run --max-rounds 1 --max-minutes 20`. Selecciona la próxima tarea elegible del grafo; no garantiza que sea esta. Verificar estado antes.

Recuperación específica

Si falla validación, no publicar brief; usar versión anterior señalando stale.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F06-07 — Revisión visual móvil/desktop, teclado, contraste y reducción movimiento; no copiar marcas.

Modalidad: interactive. **Dependencias:** F06-06. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F06.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- apps/web/src/components/accessibility.tsx
- docs/blueprint/visual-review.md

Pasos específicos

1. Playwright contra las ocho rutas propias a 390x844 y 1440x900, con prefers-reduced-motion.
2. Probar teclado, labels, foco, contraste WCAG AA, tablas pequeñas y carga lenta.
3. Comparar screenshots antes y después y requests de cada acción; no dar PASS por imagen si falló el POST.
4. Separar juicio visual humano de aserciones DOM y de negocio.

Oráculos obligatorios para el gate

- Ninguna advertencia o cifra queda fuera de pantalla sin acceso.
- Probar loading, error, empty, partial, stale y ready por ruta.
- Sin violaciones críticas de axe y con revisión visual humana documentada; no una nota 10 autoelegida.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F06-07
python3 orchestration/runner.py prepare --task F06-07 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F06-07.test.mjs
python3 orchestration/runner.py verify --task F06-07 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F06-07 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F06-07 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Corregir CSS/componentes sin sanar snapshots de dinero; congelar defecto visual con viewport/ruta.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F07-01 — Revisar amenazas RLS/storage/vectores/exports/jobs e intentar cruce tenant con IDs conocidos.

Modalidad: interactive. **Dependencias:** F06-07. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F07.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/security-findings.json
- packages/platform/src/authorization.ts

Pasos específicos

1. Ejecutar SEC-01..08 sobre DB, Storage, vectores, cache, exports y jobs locales reales, con usuarios A/B e IDs conocidos.
2. Atacar endpoints directamente; revisar exposición de service_role y SSRF mediante adjuntos.
3. Revocar acceso durante un job y repetir descarga; inspeccionar logs por PII y tokens.
4. Registrar exploit y reproducción concretos; habilitar RLS no demuestra aislamiento.

Oráculos obligatorios para el gate

- El canario SOLO_B_9F nunca aparece en respuestas, headers, logs o exports de A.
- Una función SECURITY DEFINER no elude organización ni permisos.
- Cero fugas críticas; un caso bloqueado no cuenta como PASS.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F07-01
python3 orchestration/runner.py prepare --task F07-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F07-01.test.mjs
python3 orchestration/runner.py verify --task F07-01 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F07-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F07-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Ante fuga P0: detener publicación e imports, conservar evidencia redactada y corregir con regresión antes de release.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F07-02 — Metamórficas: reordenar, duplicar, partir lotes, migrar identidad; mutación sobre dinero y autorización.

Modalidad: interactive. **Dependencias:** F07-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F07.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/economics/robustness.test.mjs
- docs/blueprint/mutation-report.json

Pasos específicos

1. Generar entradas con semilla 42: reordenamiento, duplicación, partición, monedas, nulos y BigInt grande.
2. Mutar sólo una copia temporal: sumar duplicados, null a cero, ignorar moneda u omitir tenant.
3. Ejecutar gates externos; clasificar killed, survived e invalid. Timeout no equivale a killed.
4. Cada superviviente exige una aserción de valor o justificación de equivalencia revisada.

Oráculos obligatorios para el gate

- Cambiar sólo orden o partición conserva importes.
- El mutante null a cero muere por una aserción sobre desconocido.
- Una prueba sin aserción no mejora la protección.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F07-02
python3 orchestration/runner.py prepare --task F07-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F07-02.test.mjs
python3 orchestration/runner.py verify --task F07-02 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F07-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F07-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Primero regresión, después fix; no cambiar expected porque sobrevivió un mutante.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F07-03 — Caos: worker caído, lease vencido, cuota LLM, DB caída, token revocado, dead-letter y replay.

Modalidad: interactive. **Dependencias:** F07-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F07.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- packages/jobs/src/faults.ts
- docs/blueprint/chaos-report.json

Pasos específicos

1. Inyectar en namespace fixture: crash después de commit, lease vencido, cuota, DB caída, 401, 429 y dead-letter.
2. Reproducir 3 de 3 veces con la misma semilla y reloj controlado; nunca contra producción.
3. Los oráculos DB de unicidad, cursor, presupuesto y snapshot deciden; no el LLM.
4. Registrar deadline, latencia de recuperación, backlog y errores por intento.

Oráculos obligatorios para el gate

- JOB-01/02/09/10 no pierden ni publican dos veces.
- Timeout de LLM mantiene costo uncertain.
- Revocación durante el trabajo impide publicar; replay no resucita borrados.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F07-03
python3 orchestration/runner.py prepare --task F07-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F07-03.test.mjs
python3 orchestration/runner.py verify --task F07-03 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F07-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F07-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Pausar admisión y guardar checkpoint; repetir la misma semilla tras el fix, no cambiar el orden para evitar la carrera.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F07-04 — Carga 10K/50K/150K fixtures; registrar hardware, versiones, concurrencia, p95 y costo, sin extrapolar linealmente sin medir.

Modalidad: interactive. **Dependencias:** F07-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F07.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/load-results.json
- packages/jobs/src/load-generator.ts

Pasos específicos

1. Generar datasets independientes de 10K, 50K y 150K filas con semilla y manifiesto; no mezclarlos con el smoke de totales exactos.
2. Registrar hardware, Node, DB, concurrencia, índices, EXPLAIN, p95, memoria, throughput y costo medido o no medido.
3. Empezar en 10K; ante OOM o deadline excedido, no lanzar la siguiente escala a ciegas.
4. No afirmar capacidad de 500K al mes a partir de un CSV pequeño.

Oráculos obligatorios para el gate

- accepted + rejected + duplicates + pending coincide con input en cada escala.
- Ningún agregado trunca silenciosamente en 1000 filas.
- Comparar desempeño con presupuesto aprobado; sin SLO no hay aceptación comercial de performance.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F07-04
python3 orchestration/runner.py prepare --task F07-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F07-04.test.mjs
python3 orchestration/runner.py verify --task F07-04 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F07-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F07-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Ajustar chunk y concurrencia con evidencia antes de sumar máquinas; repetir entorno y semilla conservando la corrida fallida.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F07-05 — Evaluación humano gold y sponsor de acción; ningún benchmark sustituye datos reales.

Modalidad: external. **Dependencias:** F07-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F07.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/pilot-evaluation.json

Pasos específicos

1. El operador reúne consentimientos, gold humano y sponsor, sin copiar datos reales a Git.
2. Ejecutar evaluación con holdout y ensayo cronometrado del CEO separado de latencia del pipeline.
3. Recolectar insight nuevo, acción y WTP con denominadores y respuestas autorizadas.
4. El sintético puede cerrar un ensayo técnico, no la validación comercial.

Oráculos obligatorios para el gate

- Sin entrevistas ni gold humano: blocked o not_measured.
- CEO menor a 5 minutos mide comprensión humana, no duración del job.
- Aprobar una recomendación no demuestra ahorro causal.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F07-05
python3 orchestration/runner.py prepare --task F07-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F07-05.test.mjs
python3 orchestration/runner.py verify --task F07-05 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F07-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F07-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Mantener el piloto bloqueado y avanzar sólo documentación o demo rotulada sin falsear resultados.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F07-06 — Runbooks cancelación, rollback, restore, datos borrados y eliminación de embeddings/backups conforme política.

Modalidad: interactive. **Dependencias:** F07-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F07.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/blueprint/restore-drill.json
- packages/jobs/src/deletion.ts
- supabase/migrations/0015_tombstones.sql

Pasos específicos

1. Ensayar backup y restore en entorno efímero, conciliando conteos, hashes, tenant y ledger.
2. Aplicar tombstones y retención sobre raw, texto, embeddings, caches, exports y backups; incluir derivados.
3. Rollback de app con esquema expand/contract, nunca una down-migration destructiva improvisada.
4. Medir RPO y RTO observados; no prometer SLA sin contrato.

Oráculos obligatorios para el gate

- Restore no resucita mensaje borrado ni embedding.
- Rollback conserva acceso al último snapshot válido.
- Cancelar cierra el job; sincronizadores no reingieren una entidad con tombstone.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F07-06
python3 orchestration/runner.py prepare --task F07-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F07-06.test.mjs
python3 orchestration/runner.py verify --task F07-06 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F07-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F07-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Si falla restore, bloquear release y conservar backup intacto; repetir en DB nueva, nunca resetear producción.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F08-01 — Preparar manifiesto release con SHA, migrations, config hashes, URLs y propietarios sin secretos.

Modalidad: external. **Dependencias:** F07-06. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F08.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/entrega/release-manifest.json

Pasos específicos

1. Congelar SHA, hashes de lock y configuración sin secretos, migraciones, responsables y entorno destino.
2. Exigir cero P0/P1 abiertos, ninguna prueba crítica omitida y permiso explícito para repositorio y preview.
3. Inventariar variables por nombre, uso servidor/público y rotación; no copiar valores.
4. Aplicar migraciones expand antes de la app compatible; producción requiere autorización específica.

Oráculos obligatorios para el gate

- SHA servido coincide con manifest.
- El operador valida project refs VEXA y separación de entornos.
- READY de Vercel sin smoke no es release aceptado.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F08-01
python3 orchestration/runner.py prepare --task F08-01 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F08-01.test.mjs
python3 orchestration/runner.py verify --task F08-01 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F08-01 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F08-01 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

No publicar si falta permiso o restore; conservar manifest como borrador blocked.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F08-02 — Ejecutar smoke remoto del flujo completo, dos tenants, job durable y export.

Modalidad: external. **Dependencias:** F08-01. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F08.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/entrega/smoke-remote.json

Pasos específicos

1. Desde URL autorizada, leer SHA servido, iniciar sesión A/B de ensayo e importar SYN.
2. Observar cola, worker y DB reales hasta estado terminal; navegar ocho rutas y descargar export protegido.
3. Revocar A e intentar lectura y export; inducir fallo limitado del consumidor y observar alarma.
4. Guardar trace ID, expected/observed y screenshots sin PII.

Oráculos obligatorios para el gate

- Exposición 30000 y refund 1500 coinciden en UI, API y export.
- A no lee B, antes ni después de revocar acceso.
- Apagar consumidor no convierte 202 en éxito ni pierde jobs silenciosamente.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F08-02
python3 orchestration/runner.py prepare --task F08-02 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F08-02.test.mjs
python3 orchestration/runner.py verify --task F08-02 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F08-02 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F08-02 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Rollback al SHA compatible anterior y repetir smoke; no promover sólo porque pasó CI local.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F08-03 — Ensayar pitch de 5 minutos, demo grabada de respaldo y fixture claramente rotulado si feed falla.

Modalidad: interactive. **Dependencias:** F08-02. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F08.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/entrega/demo-script.md
- docs/entrega/demo-backup-manifest.json

Pasos específicos

1. Guion de 5 minutos: problema, exposición de 300 USD, refunds de 15 USD, evidencia, acción y medición; no sumar componentes.
2. Ensayar con cronómetro humano; fallback de video local rotulado, sin afirmar conexión en vivo.
3. Mostrar portabilidad HubSpot/Zendesk mediante alias y diferencias no comparables.
4. Grabar respaldo con datos sintéticos; datos reales necesitan consentimiento y protección.

Oráculos obligatorios para el gate

- Cifras iguales al manifest y ningún ahorro inventado.
- Si falla el feed, el rótulo de respaldo y datos sintéticos permanece visible.
- El backup abre offline y su SHA queda registrado.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F08-03
python3 orchestration/runner.py prepare --task F08-03 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F08-03.test.mjs
python3 orchestration/runner.py verify --task F08-03 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F08-03 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F08-03 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Si el guion no cabe en 5 minutos, recortar contenido, no advertencias de procedencia.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F08-04 — Validar permiso de logo/datos/caso, rango financiero y supuestos antes de diapositivas.

Modalidad: external. **Dependencias:** F08-03. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXTO-CANONICO.md
- docs/blueprint/F08.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/entrega/publication-permissions.json

Pasos específicos

1. Confirmar por escrito permiso de logo, nombre, datos, caso, citas y métricas, con alcance y caducidad.
2. Separar socios propuestos de actas y 30% ofrecido de participación firmada.
3. Revisar cifras canónicas; precios y forecast son escenarios, no clientes.
4. Sin autorización usar marca VEXA y dataset ficticio, no logo de Senix.

Oráculos obligatorios para el gate

- Falta de permiso bloquea material identificable.
- Un permiso expirado o no aprobado no habilita publicar.
- NDA de acceso no equivale a permiso de publicar un caso.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F08-04
python3 orchestration/runner.py prepare --task F08-04 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F08-04.test.mjs
python3 orchestration/runner.py verify --task F08-04 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F08-04 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F08-04 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Retirar sólo el asset no autorizado del paquete, preservando la fuente privada y el registro de decisión.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F08-05 — Entregar guía usuario/operación/desarrollador, accesos revocables, backlog y fechas de próximos controles.

Modalidad: interactive. **Dependencias:** F08-04. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F08.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/entrega/README.md
- docs/entrega/GUIA-USUARIO.md
- docs/entrega/RUNBOOK.md
- docs/entrega/ACCESOS-SIN-SECRETOS.md

Pasos específicos

1. Guía por rol: login, importación, problemas, evidencia, acción, brief, export y errores.
2. Runbooks de backup, rollback, cancelación, DLQ, reconexión y revocación, con responsable y comprobación.
3. Entregar accesos revocables mediante OAuth o gestor de secretos, nunca un env por correo.
4. Backlog con IDs, severidad, reproducción, aceptación y responsable; comandos para una sesión nueva.

Oráculos obligatorios para el gate

- Un usuario de ensayo completa el flujo siguiendo sólo la guía.
- Owner revoca acceso y observa el efecto.
- El manual no anuncia funciones inexistentes; cada pendiente está rotulado.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F08-05
python3 orchestration/runner.py prepare --task F08-05 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F08-05.test.mjs
python3 orchestration/runner.py verify --task F08-05 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F08-05 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F08-05 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Actualizar README con versión entregada y SHA; no pisar manuales del cliente sin backup.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.

F08-06 — Congelar resultados y próximos experimentos: éxito del pitch no equivale a validación PMF.

Modalidad: external. **Dependencias:** F08-05. **Gate:** MISSING — escribir/revisar primero en control-plane.

Entrada (leer, no inventar contexto)

- AGENTS.md
- docs/CONTEXT0-CANONICO.md
- docs/blueprint/F08.md
- construccion/03-CONTRATOS.md

Salida prevista — no afirma que ya existe

- docs/entrega/acta-cierre.json
- docs/entrega/proximos-experimentos.md

Pasos específicos

1. Cerrar por capas: documentación, software local, demo sintética, piloto real y producción, cada una con estado y evidencia.
2. Congelar métricas, fixtures y SHA para los siguientes experimentos.
3. Acordar entrevistas, pagos de piloto y revisión a 30 días; el cierre no autoriza automatizar el negocio.
4. Archivar handoff y detener el loop comprobando procesos huérfanos.

Oráculos obligatorios para el gate

- Un pitch bien recibido no valida PMF.
- Cada bloqueo tiene responsable y condición de salida, no fecha inventada.
- Acta not_run o blocked no significa PASS; no prometer loop ilimitado.

Preparar la prueba ANTES del worker

1. Controlador escribe el gate siguiente a partir de los oráculos de arriba. Si requiere DB/browser, seguir 04-PRUEBAS-Y-LOOP.md; no sustituirlo por un JSON que diga passed.
2. Ejecutar contra implementación ausente y contra el defecto específico (nulo→cero, duplicado, fuga, según tarea). Registrar fallo por la aserción correcta; un error de setup NO mata un mutante.
3. Revisar gate fuera del candidato y hacer commit. Refrescar registro de gates y guía al agregar pruebas, antes de prepare, nunca durante una verificación.

Comandos del ciclo

```
cd ~/vexa
python3 scripts/guide.py audit
python3 scripts/guide.py packet --task F08-06
python3 orchestration/runner.py prepare --task F08-06 --max-minutes 1
# Editar sólo el worktree impreso, sin commit; guardar su ruta en CANDIDATE.
VEXA_CANDIDATE="$CANDIDATE" node --test tests/acceptance/F08-06.test.mjs
python3 orchestration/runner.py verify --task F08-06 --max-minutes 5 --approval-note "REFERENCIA A LA DECISIÓN REAL DEL OPERADOR"
# Revisión independiente de diff, controles y log; NO aceptar por autoafirmación.
# Si se rechaza: STOP + reject --task F08-06 --approval-note "referencia real"; ver capítulo 01.
python3 orchestration/runner.py accept --task F08-06 --max-minutes 5
```

Estos comandos fallan cerrados si faltan gate/dependencias/permiso; no son evidencia de ejecución. No pegar la nota de ejemplo como aprobación. La modalidad external requiere actos fuera del loop; prepare sólo abre un candidato local.

Recuperación específica

Reabrir el criterio que cambió con nueva versión y regresión; no reescribir el acta anterior.

Cierre obligatorio

task_id, baseline_sha, candidate_sha, fixture_hash, command, exit_code, expected, observed, artifacts, reviewer, pending. Las pruebas/grafos no las edita el candidato. Dos fallos agotan intento; guardar reproducción y pasar a recuperación supervisada, no editar status=accepted. Build/instalación de dependencias se hacen en copia temporal para no mutar el candidato durante el gate.